

西瓜书学习笔记

依据：周志华《机器学习》（俗称「西瓜书」）
读者定位：深度学习零基础，需要先把机器学习的语言、评价方法和经典模型搞清楚
写法约定：**讲清概念、术语和公式在说什么，不推导公式**

怎么用这份笔记

西瓜书是**机器学习通识教材**，不是深度学习专书。深度学习只是第 5 章的一部分。但后面学神经网络、过拟合、损失函数、正则化、评估指标时，几乎都会用到前几章的术语。

建议学习顺序：

阶段	章节	对深度学习的意义
必读地基	第 1 ~ 3 章	样本、过拟合、评估、线性模型、交叉熵思想
核心方法	第 4 ~ 8 章	决策树、神经网络/BP、SVM、贝叶斯、集成
无监督	第 9 ~ 10 章	聚类、PCA，常用于数据探索和预处理
进阶选读	第 11 ~ 16 章	特征选择、学习理论、半监督、图模型、规则、强化学习

公式一律用「符号含义 + 这句话在说什么」来讲，不写推导过程。

目录

第一部分 基础知识

- [第 1 章 绪论](#)
- [第 2 章 模型评估与选择](#)
- [第 3 章 线性模型](#)

第二部分 经典方法

- [第 4 章 决策树](#)
- [第 5 章 神经网络](#)
- [第 6 章 支持向量机](#)
- [第 7 章 贝叶斯分类器](#)
- [第 8 章 集成学习](#)
- [第 9 章 聚类](#)
- [第 10 章 降维与度量学习](#)

第三部分 进阶知识

- [第 11 章 特征选择与稀疏学习](#)
- [第 12 章 计算学习理论](#)
- [第 13 章 半监督学习](#)
- [第 14 章 概率图模型](#)

- [第 15 章 规则学习](#)
- [第 16 章 强化学习](#)

附录与复习

- [附录：零基础数学速查](#)
- [全书术语速查](#)
- [给深度学习零基础的衔接清单](#)
- [复习自测](#)

全书一张图：机器学习在干什么

给你一堆西瓜的数据（色泽、根蒂、敲声……），有的标了「好瓜 / 坏瓜」。机器学习要做的事是：

1. 从数据里学出一个模型 f ;
2. 希望 f 对没见过的新西瓜也能判断对。

这和「把训练集背下来」不是一回事。背下来叫**记忆**，对新样本也准才叫**学习（泛化）**。

第 1 章 绪论

- 机器学习 = 从数据中学出对未知样本也有效的模型。
- 必须分清：样本、特征、标记、训练集、假设空间、归纳偏好。
- **没有免费的午餐**：不存在「处处最优」的算法，算法是否好取决于问题是否匹配它的偏好。

1.1 机器学习是什么

机器学习（Machine Learning）：通过计算的手段，利用经验（数据）来改善系统自身的性能。

更具体一点：计算机从数据中产生**模型（model）**，再用模型对**新数据**给出判断。

三个关键词：

- **数据**：经验的数字化。
- **模型**：学到的那个「判断规则」或「函数」。
- **泛化**：对新样本仍然有效，而不是只对训练数据有效。

1.2 基本术语

用「判断西瓜好不好」把术语一次对齐。

术语	英文	直观理解
数据集	data set	很多条西瓜记录放在一起
示例 / 样本	instance / sample	一条记录，一个西瓜
属性 / 特征	attribute / feature	色泽、根蒂、敲声等描述维度

术语	英文	直观理解
属性值	attribute value	「青绿」「浊响」这些具体取值
属性空间 / 样本空间 / 输入空间	attribute space	所有可能属性组合构成的空间
特征向量	feature vector	把一个样本写成向量 $\boldsymbol{x} = (x_1, x_2, \dots, x_d)$
维数	dimensionality	特征个数 d
标记 / 标签	label	好瓜还是坏瓜
样例	example	「样本 + 标记」, 即 $(\boldsymbol{x}, y)$
标记空间 / 输出空间	label space	所有可能标记的集合
分类	classification	预测离散类别, 如好瓜/坏瓜
回归	regression	预测连续值, 如含糖率
二分类	binary classification	只有两类
多分类	multi-class classification	三类及以上
聚类	clustering	没有标记, 把相似样本聚到一堆
监督学习	supervised learning	训练数据有标记
无监督学习	unsupervised learning	训练数据无标记
训练集	training set	用来学模型的数据
测试集	test set	用来估计模型对新样本表现的数据
假设	hypothesis	从数据学出的那个模型, 记为 f
学习算法	learning algorithm	从数据得到模型的那套办法
预测	prediction	把 $\boldsymbol{x}$ 送进 f , 得到 $\hat{y} = f(\boldsymbol{x})$

样本通常写成:

$$\boldsymbol{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})$$

- i : 第几个样本
- d : 特征维数
- x_{ij} : 第 i 个样本的第 j 个特征

带标记的样例: $(\boldsymbol{x}_i, y_i)$ 。整个训练集:

$$D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \dots, (\boldsymbol{x}_m, y_m)\}$$

m 是样本个数。

分类 vs 回归

- 分类： y 是类别。西瓜好/坏、数字 0~9、猫/狗。
- 回归： y 是实数。房价、温度、含糖率。

监督学习 vs 无监督学习

- 监督：有 y ，学 $x \rightarrow y$ 。
- 无监督：没有 y ，学数据本身的结构（聚类、降维）。
- 半监督：少量有标记、大量无标记。
- 强化学习：通过与环境交互、拿奖赏来学策略（第 16 章）。

1.3 假设空间

学习可以看成：在所有可能的模型里，找出与训练集一致的那个（或那些）。

- **假设空间 (hypothesis space)**：学习算法能考虑的所有可能模型组成的集合，常记为 $\mathcal{H}$ 。
- **版本空间 (version space)**：假设空间里，与当前训练集**一致**的那些假设。

直观例子：如果只允许用「色泽、根蒂」做判断，所有「如果色泽=...且根蒂=...则好瓜」的规则，就构成假设空间。训练数据会把明显矛盾的规则删掉，剩下的就是版本空间。

数据往往不够把版本空间缩到只剩一个假设，所以「学出一个模型」通常还要靠下一节的**归纳偏好**。

1.4 归纳偏好

归纳偏好 (inductive bias)：学习算法在多个「都能拟合训练数据」的模型里，更喜欢哪一种。

没有偏好，算法就不知道选哪一个。例如：

- 决策树偏好「更简单、更短」的树；
- 线性模型偏好「直线/平面」这种简单分界；
- 神经网络偏好由网络结构和训练方式所隐含的那类函数。

奥卡姆剃刀 (Occam's razor)：若多个假设与观测一致，选最简单的那个。这是一种常见偏好，但「简单」本身也要定义。

没有免费的午餐定理 (NFL, No Free Lunch)：

对所有可能的问题平均起来，任何两个学习算法的期望性能相同。

白话：不存在「永远最好」的算法。算法 A 在这类问题上强，往往在另一类问题上弱。所以：

- 要看数据和任务是否匹配算法的偏好；
- 实际中必须用评估方法（第 2 章）在你的问题上比较模型。

1.5 ~ 1.6 发展与应用

知道机器学习从早期符号主义、连接主义、统计学习走到今天的深度学习即可。应用覆盖视觉、语音、推荐、医疗、金融等。对入门更重要的是术语和后面的评估、模型，而不是年表。

第 1 章小结

你要能用自己的话回答
样本、特征、标记分别是什么？
分类和回归差在哪？ 监督和无监督差在哪？
什么是假设空间、归纳偏好？
为什么「没有免费的午餐」意味着必须针对具体任务选模型？

第 2 章 模型评估与选择

学深度学习之前，这一章比任何网络结构都重要。核心就三句话：

- 1. 训练误差小不等于模型好，要看**泛化误差**。
- 2. 必须用合理的方法划分训练/验证/测试，否则评估是自欺欺人。
- 3. 不同任务要用不同指标：准确率、精确率/召回率、F1、ROC/AUC。

2.1 经验误差与过拟合

对分类，**错误率**：

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) \neq y_i)$$

- $\mathbb{I}(\cdot)$ ：指示函数，括号里为真取 1，否则取 0。
- 这句话：在数据集 D 上，预测错的比例。

精度 (accuracy)： $1 - E(f; D)$ ，即预测对的比例。

两个关键词：

- **经验误差 / 训练误差**：模型在**训练集**上的误差。
- **泛化误差**：模型在**新样本（总体分布）**上的误差。我们真正关心的是它。

训练误差可以算，泛化误差不能直接算，所以要用测试集去**估计**它。

过拟合 (overfitting)：把训练数据学得**太「死」**，连噪声都记住了，训练误差很低，但泛化很差。

欠拟合 (underfitting)：模型太简单，连训练数据都没学好。

	训练误差	泛化/测试误差	常见原因
欠拟合	高	高	模型太弱、训练不够、特征不足
合适	较低	较低	复杂度与数据匹配
过拟合	很低	高	模型太强、数据太少、训练过久

缓解过拟合的常见思路（后面章节会反复出现）：加数据、简化模型、正则化、早停、剪枝、Dropout 等。

欠拟合则要反过来：换更强的模型、加特征、减少过度正则、多训练一会儿。

2.2 评估方法

测试集必须尽量**不参与训练**，否则评估偏乐观。

留出法 (hold-out)

把数据分成两互斥部分：训练集 + 测试集。常用比例 7:3 或 8:2。

注意：

- 划分要尽量保持类别比例，这叫**分层采样**。
- 单次划分偶然性大，可多次随机划分取平均。

交叉验证法 (cross validation)

k 折交叉验证：把数据分成 k 份，每次用 $k - 1$ 份训练、1 份测试，轮转 k 次，把 k 次结果平均。

常用 $k = 10$ 。

留一法 (LOO, Leave-One-Out)： $k = m$ ，每次只留 1 个样本做测试。评估几乎无偏，但计算极贵。

自助法 (bootstrapping)

有放回地抽样 m 次，得到大小仍为 m 的训练集。没被抽到的样本当测试集，大约有 36.8% 的样本会落在测试集（因为 $(1 - 1/m)^m \approx 1/e \approx 0.368$ ）。

适合数据集很小、留出法会让训练集再缩一截的情况。缺点是改变了初始数据分布。

调参与最终模型

调超参数（学习率、树深度、正则系数……）时，不能反复拿**测试集**来选最好的那组参数，否则测试集变成了「变相训练集」，评估会虚高。

正确做法：

- **训练集**：学参数（权重）。
- **验证集 (validation set)**：选超参数、做早停。
- **测试集**：最终只评估一次，报告泛化表现。

选定超参数后，可用全部训练数据（含验证集）再训练一次，得到最终模型。

2.3 性能度量

「误差小」要看任务。分类、回归、排序用的尺子不同。

回归常用

均方误差 (MSE)：

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m (f(\mathbf{x}_i) - y_i)^2$$

含义：预测值与真值差的平方的平均。差得越远，惩罚越大。

分类：错误率与精度

见 2.1。类别很不均衡时，精度会骗人：100 个样本里 99 个负例，模型全猜负例，精度也有 99%。这时要用下面的精确率/召回率。

查准率、查全率与 F1

先看混淆矩阵（以正类/负类为例）：

	预测为正	预测为负
真实为正	TP（真正例）	FN（假反例）
真实为负	FP（假正例）	TN（真反例）

- **查准率 / 精确率 Precision**：预测为正的里面，有多少真是正的。

$$P = \frac{TP}{TP + FP}$$

- **查全率 / 召回率 Recall**：所有真正的正例里，找出了多少。

$$R = \frac{TP}{TP + FN}$$

直观：查准率怕「报假喜」，查全率怕「漏报」。查垃圾邮件时提高查准率；查癌症时更在乎查全率。

二者常常此消彼长。综合指标 **F1**：

$$F1 = \frac{2PR}{P + R}$$

它是 P 和 R 的调和平均，两者都高 F1 才高。

若更看重精确率或召回率，可用 F_β ： $\beta > 1$ 更看重召回， $\beta < 1$ 更看重精确率。

多组 P, R 可画 **P-R 曲线**。曲线下面积越大通常越好。两条曲线谁「包住」谁，谁更好；相交则看平衡点（Break-Even Point, $P = R$ 时的取值）或 F1。

ROC 与 AUC

很多模型输出的是「属于正类的概率」或分数，要设一个阈值才变成 0/1。阈值不同， P, R 不同，TPR/FPR 也不同。

- **TPR（真正例率）**： $TPR = TP / (TP + FN)$ ，就是召回率。
- **FPR（假正例率）**： $FPR = FP / (TN + FP)$ ，负例被误报为正的的比例。

ROC 曲线：横轴 FPR，纵轴 TPR。从左下到右上。越靠近左上角越好。

AUC（Area Under ROC Curve）：ROC 曲线下的面积。取值大约在 0.5 ~ 1。

- AUC = 1：完美排序。
- AUC = 0.5：相当于随机猜。
- AUC 的直观含义：随机抽一个正例和一个负例，模型给正例打分更高的概率。

ROC 相对 P-R 对类别不平衡更不敏感，所以不平衡分类里 AUC 很常用。

代价敏感

有时「把病人判成健康」比「把健康人判成病人」代价大得多。可把不同错误赋不同代价，用**代价矩阵**算代价敏感错误率，或画**代价曲线**。知道「指标要反映真实损失」即可。

2.4 比较检验

两次实验精度差 0.5%，可能只是划分偶然，不一定真的更好。统计检验用来问：这个差异靠不靠谱？

零基础先记住结论：

- 单次结果不够，要多次实验；
- 常用：成对 t 检验、McNemar 检验（比较两个分类器在同一测试集上的对错差异）；
- 多个算法在多个数据集上比较，可用 Friedman 检验 + Nemenyi 后续检验。

入门阶段会看实验表格即可，不必先把检验公式背下来。

2.5 偏差与方差

把泛化误差拆开看，能解释欠拟合/过拟合。

对回归任务，泛化误差可分解为：

$$E(f; D) = \text{bias}^2(x) + \text{var}(x) + \varepsilon^2$$

三个词：

- **偏差 (bias)**：模型平均预测与真实标记的差距。偏差大 ≈ 模型假设太简单，欠拟合。
- **方差 (variance)**：换一批训练集，模型预测抖得厉不厉害。方差大 ≈ 对数据太敏感，过拟合。
- **噪声 ε** ：数据本身标错、测不准，学不掉的部分。

直观权衡：

- 模型太简单：偏差大、方差小。
- 模型太复杂：偏差小、方差大。
- 训练到刚刚好：两者比较均衡。

深度学习里网络很强，偏差往往较小，更要靠数据、正则、早停来压方差。

第 2 章小结

必懂清单
经验误差 ≠ 泛化误差；过拟合 / 欠拟合
留出法、k 折交叉验证、验证集用来调参
混淆矩阵、P、R、F1、ROC、AUC
偏差大 ≈ 欠拟合，方差大 ≈ 过拟合

第 3 章 线性模型

线性模型是最简单的「可学习机器」，也是神经网络的一块积木：一层全连接、一个加权求和，就是线性模型。

基本形式：

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b = \mathbf{w}^\top \mathbf{x} + b$$

- $\mathbf{w}$ ：权重，每个特征的重要性。
- b ：偏置（截距），没有特征时的基准输出。
- $\mathbf{w}^\top \mathbf{x}$ ：特征和权重的内积（加权求和）。

学线性模型 = 学一组 $\mathbf{w}, b$ 。

3.2 线性回归

任务：预测连续值 y 。希望 $f(\mathbf{x}) \approx y$ 。

常用均方误差当损失：

$$E(\mathbf{w}, b) = \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2$$

含义：每个样本「预测 - 真实」的平方加起来，越小越好。

当数据矩阵 $X^\top X$ 可逆时，最小二乘有闭式解：

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$$

这句话：有一个公式能直接算出最优权重（在 MSE 意义下）。实际中维数高或不可逆时，用梯度下降等数值方法，思想和深度学习训练一样。

多元线性回归：多个特征。

广义线性模型：先对 $\mathbf{w}^\top \mathbf{x} + b$ 做非线性包裹，例如对数几率回归就是其中一种。

3.3 对数几率回归

名字叫回归，其实是**分类**算法。

把线性函数的输出压到 $(0, 1)$ ，当作「正类概率」：

$$y = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

这个 S 形函数叫 **Sigmoid / 对数几率函数**。

- 输出接近 1：很像正类。
- 输出接近 0：很像负类。
- 0.5 附近：不确定。

变形一下：

$$\ln \frac{y}{1-y} = \mathbf{w}^\top \mathbf{x} + b$$

左边叫 **几率 (odds)** 的对数：正类概率与负类概率之比再取 log。所以叫对数几率回归 (logistic regression)。

训练时常用**极大似然**：让模型给真实类别的概率尽量大，这和深度学习里的**交叉熵损失**是同一件事。

零基础记住：

- 线性回归：输出实数。
- 对数几率回归：线性输出 → Sigmoid → 概率 → 分类。
- 神经网络二分类的最后一层，经常就是这个结构。

3.4 线性判别分析 LDA

LDA (Linear Discriminant Analysis)：一种经典线性分类思想，也可作降维。

想法：把样本投影到一条直线上，使得

- 同类尽量近（类内散度小）；
- 异类尽量远（类间散度大）。

目标可写成：

$$J = \frac{\mathbf{w}^\top S_b \mathbf{w}}{\mathbf{w}^\top S_w \mathbf{w}}$$

- S_b ：类间散度矩阵（两类中心差得远不远）。
- S_w ：类内散度矩阵（每一类内部散不散）。
- J 越大：投影后越好分。

和 PCA（第 10 章）对比：PCA 找「方差最大」的方向（不管类别）；LDA 找「最好分类」的方向（用标记）。

3.5 多分类学习

二分类器怎么处理 N 类？常见拆法：

策略	做法	要训练几个分类器
OvR（一对剩余）	每类 vs 其余所有类	N 个
OvO（一对一）	每两类之间训练一个	$N(N - 1)/2$ 个
MvM（多对多）	若干类 vs 若干类，如纠错输出码	取决于编码

预测时：OvR 看谁的分数最高；OvO 投票。类别很多时 OvO 分类器更多，但每个问题更小。深度学习里更常见的是：**最后一层 N 个输出 + Softmax**，一次解决多分类。

Softmax（书中后文/实践常用，这里提前对齐术语）：

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

把一组分数变成和为 1 的概率。二分类时 Softmax 与 Sigmoid 本质相通。

3.6 类别不平衡

正例远少于负例时，模型会偷懒全猜大类。

基本对策：**再缩放 (rescaling)**

$$\frac{y'}{1-y'} = \frac{y}{1-y} \times \frac{m^+}{m^-}$$

含义：按正负样本数量比，把决策阈值从 0.5 挪开。

实践中还有：

- 欠采样多数类、过采样少数类（如 SMOTE）；
- 给少数类更大的损失权重；
- 不用精度，改看 F1、AUC、召回率。

第 3 章小结

模型	输出	一句话
线性回归	实数	用直线拟合
对数几率回归	概率	Sigmoid + 分类
LDA	投影后分类	类间远、类内近
OvR / OvO	多类	用多个二分类器拼起来

第 4 章 决策树

决策树用一串「if-then」做判断，像流程图。每个内部结点问一个属性，每个分支是属性的一种取值，叶子给出类别。

经典算法：ID3、C4.5、CART。差别主要在**用什么指标选划分属性**，以及是否能处理连续值、后剪枝等。

基本流程：递归地选属性把数据切开，直到

- 结点中样本全属一类；或
- 属性用完 / 无法再划分；或
- 样本太少。

4.2 划分选择

好的划分：切开后，每一堆尽量「纯」。纯度用信息熵或基尼指数衡量。

信息熵

$$\text{Ent}(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

- p_k : 数据集 D 中第 k 类的比例。
- 熵越大越混乱。两类各占 50% 时熵最大；全是一类时熵为 0。

信息增益 (ID3)

$$\text{Gain}(D, a) = \text{Ent}(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Ent}(D^v)$$

- a : 候选属性, 有 V 个取值。
- D^v : 属性 a 取第 v 个值的那些样本。
- 含义: 用属性 a 划分后, 熵下降了多少。下降越多, 这个属性越有用。

缺点: 取值特别多的属性 (如「学号」) 信息增益会虚高, 因为切得很碎、每堆都很纯。

增益率 (C4.5)

$$\text{Gain_ratio}(D, a) = \frac{\text{Gain}(D, a)}{\text{IV}(a)}$$

其中固有值

$$\text{IV}(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

属性取值越多, $\text{IV}(a)$ 越大, 从而抑制「切得太碎」。C4.5 常先筛信息增益高于平均的属性, 再选增益率最高的。

基尼指数 (CART)

$$\text{Gini}(D) = 1 - \sum_{k=1}^{|Y|} p_k^2$$

基尼越小越纯。属性 a 的基尼:

$$\text{Gini_index}(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Gini}(D^v)$$

选基尼指数最小的属性。CART 的分类树用基尼, 回归树用平方误差; CART 还是**二叉树** (每次问 yes/no) 。

4.3 剪枝

树太大极易过拟合。剪枝是决策树的主要抗过拟合手段。

- **预剪枝**: 划分前先看, 若在验证集上不能带来提升, 就停止生长。快, 但可能目光短浅 (这一刀现在没用, 后面有用) 。
- **后剪枝**: 先长成完整树, 再从下往上砍。通常泛化更好, 但训练更慢。

4.4 连续值与缺失值

- **连续值**：把连续属性排序，尝试不同阈值（如相邻取值的中点），变成「 $\leq t$ / $> t$ 」这种二分。
- **缺失值**：选属性时按无缺失样本计算，再用有缺失样本落到各分支的比例加权；预测时一个样本可按权重走多条路。

4.5 多变量决策树

普通决策树每次用一个属性，分界与坐标轴平行。多变量决策树在结点上用属性的线性组合，分界可以是斜的，更灵活，但更难解释。

第 4 章小结

算法	划分准则	特点
ID3	信息增益	偏好取值多的属性
C4.5	增益率	能处理连续值、缺失值
CART	基尼 / 平方误差	二叉树，可做分类或回归

随机森林、GBDT、XGBoost 都是在决策树之上做集成（第 8 章）。

第 5 章 神经网络

神经网络 = 很多「神经元」连成多层，用误差逆传播（BP）调权重。今天的深度学习是这一思想在「更深、更宽、更多数据、更好训练技巧」上的发展。

5.1 神经元模型

M-P 神经元：

$$y = f\left(\sum_{i=1}^n w_i x_i - \theta\right)$$

- x_i ：输入。
- w_i ：连接权重。
- θ ：阈值。常把 $-\theta$ 写成偏置 b ，即 $f(\mathbf{w}^\top \mathbf{x} + b)$ 。
- f ：激活函数，把加权和变成输出。

理想阶跃函数太硬、不可导，常用 Sigmoid：

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

输出在 (0,1)，平滑可导。现代深度学习还常用 ReLU： $f(x) = \max(0, x)$ ，计算简单、深层更好训。

5.2 感知机与多层网络

感知机 (perceptron)：一层神经元，只能处理**线性可分**问题（能用一条直线分开）。异或（XOR）就分不开。

多层前馈网络：输入层 → 若干隐层 → 输出层。层与层之间全连接，同层不连，也不跨层回边。这就是最基本的深度网络骨架。

两个重要事实（记住结论即可）：

- 只要隐层够宽，多层网络能逼近任意复杂的连续函数（万能近似）。
- 实践中不是「越宽越好」，而是深度、结构、数据和训练一起决定效果。

5.3 误差逆传播 BP

BP (Backpropagation, 误差逆传播) 是训练多层网络最常用的算法，本质是**链式法则算梯度 + 梯度下降更新权重**。

常用均方误差（也可换成交叉熵，分类时更常见）：

$$E_k = \frac{1}{2} \sum_{j=1}^{\ell} (\hat{y}_j^k - y_j^k)^2$$

对一个样本，输出和真实差多少。全体样本再求和或求平均。

梯度下降更新：

$$v \leftarrow v + \Delta v, \quad \Delta v = -\eta \frac{\partial E}{\partial v}$$

- v ：某个权重或偏置。
- η ：学习率，步子迈多大。太大震荡，太小学得慢。
- $\partial E / \partial v$ ：误差对这个参数的梯度，表示「参数往哪边调误差下降」。

BP 的流程直觉：

1. **前向传播**：输入一层层算到输出，得到预测。
2. **算误差**：预测和真实差多少。
3. **反向传播**：从输出层往回传梯度，每层都知道自己该怎么改权重。
4. **更新参数**：沿负梯度走一小步。
5. 重复直到收敛。

标准 BP 往往「每来一个样本就更新一次」，叫随机/在线方式。也可以一批样本算平均梯度再更新，叫**批量 / 小批量 (mini-batch)**，这是现在深度学习的默认做法。

学习率衰减、动量等是为了让训练更稳。西瓜书还提到在目标里加正则项 $\lambda \|\mathbf{w}\|^2$ 减轻过拟合，这就是**权重衰减 (weight decay)**。

5.4 全局最小与局部极小

误差曲面坑坑洼洼：

- **全局最小**：所有参数里误差最低的点。

- **局部极小**：周围都比它高，但别处可能有更深的坑。

跳出不好的局部区域的常用策略：

- 多组不同随机初始化，取最好；
- 模拟退火式的「先允许变差再逐渐变严」；
- 随机梯度本身的噪声有时能帮你跳出浅坑。

深度学习实践里，更常遇到的是**鞍点**（有的方向上坡、有的方向下坡）和训练不稳定，优化器（SGD、Adam）和学习率调度因此很重要。

5.5 其他常见神经网络

名称	一句话
RBF 网络	隐层用径向基函数（距离中心越远响应越小）
ART 网络	竞争学习，可增量学新类
SOM	自组织映射，把高维数据铺到低维网格上，无监督
级联相关	动态加神经元，结构也在学
Elman 网络	带反馈，能处理序列，是 RNN 的前辈
Boltzmann 机 / RBM	基于能量的随机网络，深度信念网的构件

这些在教材史里重要；当前主流工程主要是 CNN、RNN/Transformer、残差网络等，但「层、激活、损失、反向传播」的骨架没变。

5.6 深度学习

深度学习：很多隐层的神经网络，以及让深层网络真正能训起来的一套技术。

为什么深层难训、后来又可行？记住几点：

- 层数一多，误差来回传会**消失或爆炸**（梯度变得极小或极大）。
- 无监督预训练（逐层训练 RBM/自编码器再微调）是早期解药。
- 后来更有效的是：ReLU、更好初始化、BatchNorm、残差连接、Dropout、大量数据、GPU。

表示学习角度：浅层学边、角，深层学物体部件，再高层学类别。不必手设计全部特征。

过拟合在深度模型里更常见，因为参数极多。对策：Dropout、数据增强、正则、早停、更大的数据。

第 5 章小结

神经元加权求和 + 激活 → 多层堆叠 → BP 用梯度更新权重。

学深度学习时，把这一章和「损失、学习率、过拟合、验证集」绑在一起，就接得上后续的 CNN/Transformer。

第 6 章 支持向量机

SVM 要找一条「尽量宽」的分界带，让两类分得最开。真正起作用的往往只是贴在边界上的那几个点，叫**支持向量**。

线性不可分时：软间隔（允许少量犯错）+ 核函数（隐式升维）。

6.1 间隔与支持向量

给定训练集

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}, \quad y_i \in \{+1, -1\}$$

对线性可分的二分类，分界是超平面：

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

- $\mathbf{w}$ ：法向量，决定超平面方向；
- b ：位移项，决定超平面与原点的距离。

样本点 $\mathbf{x}_i$ 到超平面的几何间隔：

$$r_i = \frac{|\mathbf{w}^\top \mathbf{x}_i + b|}{\|\mathbf{w}\|}$$

距离超平面最近的那些样本点叫**支持向量**。两个异类支持向量到超平面的距离之和，称为**间隔 (margin)**：

$$\gamma = \frac{2}{\|\mathbf{w}\|}$$

最大化间隔，等价于在把样本正确分开的前提下最小化 $\|\mathbf{w}\|$ 。间隔越宽，通常越抗扰动。

硬间隔 SVM 的优化目标：

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

- 约束：每个样本都在间隔带的「正确一侧」，至少距离到函数间隔为 1。
- 目标： $\|\mathbf{w}\|$ 越小，几何间隔 $2/\|\mathbf{w}\|$ 越大。

落在间隔边界上的样本就是支持向量。决策面只由它们决定，删掉远处的点，超平面不变。

6.2 对偶问题

原问题有约束，常转成对偶问题来解。对偶后，模型可写成训练样本的线性组合：

$$f(\mathbf{x}) = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$$

- α_i ：每个样本的拉格朗日乘数。
- 只有支持向量的 $\alpha_i > 0$ ，其他为 0。

实用含义：预测时主要和少数支持向量做内积，这为核函数铺路。

6.3 核函数

有些数据在原始空间直线分不开，映射到高维就分开了。显式写出 $\phi(\mathbf{x})$ 可能维数极高甚至无穷维。

核函数直接算高维内积，不必写出映射本身：

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

这就是**核技巧 (kernel trick)**：凡是算法只用到样本内积，就可以把内积换成核，得到非线性版本（核化），从而避免高维爆炸。

常见核：

核	形式直觉	适用
线性核	就是原始内积	特征已经很多、近似可分
多项式核	特征的多项式组合	中等非线性
高斯核 / RBF	$\exp(-\ \mathbf{x}_i - \mathbf{x}_j\ ^2 / (2\sigma^2))$	很灵活，最常用
拉普拉斯核、Sigmoid 核	其他相似度	特定场合

高斯核的 σ 很敏感：太小易过拟合，太大趋近线性。

6.4 软间隔与正则化

现实数据有噪声、重叠，硬间隔会无解或过拟合。**软间隔 (soft margin)** 允许一些点进间隔带甚至分错，用松弛变量 $\xi_i \geq 0$ 表示「犯了多少规」。

优化目标：

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

$C > 0$ 是惩罚系数：

- C 大：少容错，更贴近硬间隔，易过拟合；
- C 小：更容忍错误，间隔更宽，可能欠拟合。

这和正则化是同一类思想：第一项要简单（间隔大），第二项要拟合数据。 C 就是在两者之间做权衡。

6.5 支持向量回归 SVR

SVM 不仅做分类，还可以做回归，称为**支持向量回归 (SVR)**。

传统回归要求预测尽量等于真实标签；SVR 允许存在一条宽度为 ϵ 的间隔带：样本落在带内不计损失，只有落到带外才惩罚。

SVR 优化目标：

$$\min_{\mathbf{w}, b, \xi, \hat{\xi}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m (\xi_i + \hat{\xi}_i)$$

约束：

$$\begin{aligned}f(\mathbf{x}_i) - y_i &\leq \epsilon + \xi_i \\y_i - f(\mathbf{x}_i) &\leq \epsilon + \hat{\xi}_i \\ \xi_i &\geq 0, \hat{\xi}_i \geq 0\end{aligned}$$

- ξ_i 、 $\hat{\xi}_i$ ：上下两个方向的松弛变量（预测偏高 / 偏低）；
- ϵ ：间隔带半宽，越大越「宽容」。

引入拉格朗日乘子得到对偶形式后，同样可以换成核函数做非线性回归。最终解仍然只由部分支持向量决定。

6.6 核方法

不只是 SVM：任何只用到样本内积的算法，都可以把内积换成核函数，从而拓展到非线性场景。

核心做法：把 $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ 替换成 $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ 。

常见例子：

- **核 PCA**：把 PCA 核化，做非线性降维；
- **核 LDA**：把线性判别分析核化。

SVM 是核方法最成功的代表。

第 6 章小结

SVM = 最大间隔 + 支持向量 + （必要时）核函数与软间隔。

和神经网络比：SVM 在中小规模、特征明确时很强且理论清晰；大规模端到端特征学习目前深度网络更占优。

第 7 章 贝叶斯分类器

先根据数据估计「各类别有多常见、各类别下特征长什么样」，再对未知样本算它最可能属于哪一类。理论基础是贝叶斯定理。

0-1 损失下，这等价于选**后验概率最大**的类。

7.1 贝叶斯决策论

给定 N 个类别。 λ_{ij} 表示：真实类别是 c_j ，却预测成 c_i 时的**损失**。

样本 $\mathbf{x}$ 被判为类别 c_i 的**条件期望损失（条件风险）**：

$$R(c_i | \mathbf{x}) = \sum_{j=1}^N \lambda_{ij} P(c_j | \mathbf{x})$$

贝叶斯最优决策：对每个 $\mathbf{x}$ ，选出使条件风险最小的类别。

$$h^*(\mathbf{x}) = \arg \min_{c_i \in \mathcal{Y}} R(c_i | \mathbf{x})$$

0-1 损失（错分损失）：判错就损失 1，判对损失 0，即 $\lambda_{ij} = 1 \ (i \neq j)$ ， $\lambda_{ii} = 0$ 。此时

$$R(c_i | \mathbf{x}) = 1 - P(c_i | \mathbf{x})$$

最小化风险就等价于**最大化后验概率**：

$$h^*(\mathbf{x}) = \arg \max_{c_i} P(c_i | \mathbf{x})$$

这就是**贝叶斯最优分类器**。它的错误率叫**贝叶斯错误率**，是所有分类器的理论上限（数据本身可能重叠，最好也只能到这儿）。

后验由贝叶斯公式计算：

$$P(c | \mathbf{x}) = \frac{P(c) P(\mathbf{x} | c)}{P(\mathbf{x})}$$

- $P(c)$ ：**先验**，没看样本前，类别 c 有多常见。
- $P(\mathbf{x} | c)$ ：**似然**，这个类里出现特征 $\mathbf{x}$ 的可能性。
- $P(\mathbf{x})$ ：**证据**，特征 $\mathbf{x}$ 本身有多常见（对所有类一样，比较时可忽略）。
- $P(c | \mathbf{x})$ ：**后验**，看到 $\mathbf{x}$ 之后属于 c 的概率。

7.2 极大似然估计 MLE

很多时候要先估计 $P(\mathbf{x} | c)$ 里的参数。假定类别 c 的样本服从某参数化分布 $P(\mathbf{x} | c; \theta_c)$ 。

极大似然 (MLE)：选让「当前这批数据出现概率最大」的参数。

似然：给定参数时，训练集出现的概率。常用对数似然：

$$LL(\theta_c) = \log P(D_c | \theta_c) = \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} | \theta_c)$$

极大似然估计就是：

$$\hat{\theta}_c = \arg \max_{\theta_c} LL(\theta_c)$$

例子：抛硬币 100 次里 70 次正面，MLE 认为正面概率是 0.7。深度学习里用交叉熵训练，也是在做极大似然。

要点：

1. MLE 是频率学派思想，把参数当作「确定但未知」的常量；
2. 依赖**分布假设**；假设错了，估计会很差；
3. 高斯分布是最常用的假设。

对比：贝叶斯估计把参数也看作随机变量，先给参数一个先验，再求参数的后验。

7.3 朴素贝叶斯

直接估 $P(\mathbf{x} | c)$ 很难，因为 $\mathbf{x}$ 是很多特征的组合，组合数爆炸。

朴素 (naive) 假设：给定类别后，特征之间相互独立。于是

$$P(c | \mathbf{x}) \propto P(c) \prod_{i=1}^d P(x_i | c)$$

- d : 属性（特征）个数；
- x_i : 样本第 i 个属性的取值。

每个特征单独估一个 $P(x_i | c)$ 即可。这就是垃圾邮件过滤、文本分类的经典做法。

代入后得到朴素贝叶斯分类器：

$$h_{nb}(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} P(c) \prod_{i=1}^d P(x_i | c)$$

参数估计

1. 先验概率

$$P(c) = \frac{|D_c|}{|D|}$$

D_c : 训练集中属于类别 c 的样本集合。

2. 离散属性的条件概率

$$P(x_i | c) = \frac{|D_{c,x_i}|}{|D_c|}$$

D_{c,x_i} : 类别为 c 、且第 i 个属性取值为 x_i 的样本集合。

3. 连续属性

假设每个属性在该类下服从高斯分布，用 MLE 估计均值、方差：

$$p(x_i | c) = \mathcal{N}(x_i | \mu_{c,i}, \sigma_{c,i}^2)$$

拉普拉斯平滑

若某个取值在某类训练数据里从没出现， $P(x_i | c)$ 会变成 0，整条乘积为 0，这叫**零概率问题**。用**拉普拉斯平滑**：计数时先给每个取值加 1（伪计数）。

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + |\mathcal{Y}|}$$

$$\hat{P}(x_i | c) = \frac{|D_{c,x_i}| + 1}{|D_c| + N_i}$$

- $|\mathcal{Y}|$: 类别总数；
- N_i : 第 i 个属性有多少种可能取值。

三种用法

1. **预测**: 直接算后验，输出类别；
2. **输出概率**: 输出各类后验概率；
3. **模型存储**: 保存所有 $P(c)$ 和各个 $P(x_i | c)$ ，新样本来了直接算。

优点：简单、算得快，小数据集上也常能用。

缺点：**条件独立假设现实中很难成立**，多数时候是强行近似，但仍然很好用。

7.4 半朴素贝叶斯

朴素贝叶斯假设全部属性互相独立，往往太强。半朴素贝叶斯做折中：**允许部分属性之间存在依赖**。

核心思路：每个属性最多再依赖另外一个父属性。

SPODE (Super-Parent One-Dependent Estimator)

指定一个「超父」属性，其他属性都依赖它：

$$P(\mathbf{x} | c) = P(x_1 | c) \prod_{i=2}^d P(x_i | c, x_1)$$

x_1 为超父；每个属性同时依赖类别 c 和超父。缺点：超父不好选。

AODE (Averaged One-Dependent Estimator)

把每个属性轮流当超父，再把多个 SPODE 的结果平均，从而避开「选哪个超父」：

$$P(c | \mathbf{x}) \propto \sum_{i=1}^d P(c, x_i) \prod_{j=1, j \neq i}^d P(x_j | c, x_i)$$

- 为降低开销，一般只用样本数足够多的候选超父；
- AODE 通常不必调参，效果往往优于朴素贝叶斯。

TAN (Tree-Augmented Naive Bayes, 树增强朴素贝叶斯)

用**互信息**刻画属性之间的依赖，再建成一棵最大带权生成树：

1. 算两两属性的条件互信息 $I(x_i, x_j | c)$ ，衡量「给定类别后这两个属性有多依赖」；
2. 以互信息为边权，构建最大生成树；
3. 类别 c 仍是所有属性的公共父结点，树再表达属性之间的额外依赖。

每个属性除了类别外最多再有一个父结点，复杂度可控。

对照：

- **NB**：所有属性条件独立；
- **AODE**：多个独依赖模型取平均；
- **TAN**：用一棵树刻画属性依赖。

7.5 贝叶斯网

用有向无环图描述变量之间的条件依赖：箭头表示直接影响。可表示比朴素贝叶斯更复杂的依赖，用于推断「观察到一部分变量后，其他变量的概率」。

学习分两块：学图结构（很难）、学每张表里的概率参数。推断一般也是 NP 难，实际用近似。

7.6 EM 算法

用来解决：**存在未观测隐变量 (latent variable) 时的参数估计**。

记观测数据为 X ，隐变量为 Z （缺失或不可观测），模型参数为 Θ 。完整数据的对数似然是 $\ln P(X, Z | \Theta)$ ，但 Z 看不见。EM 用两步迭代逼近：

E 步 (Expectation, 期望)：固定当前参数 Θ^t ，在隐变量后验 $P(Z | X, \Theta^t)$ 下，对完整对数似然求期望，得到 Q 函数：

$$Q(\Theta | \Theta^t) = \mathbb{E}_{Z|X, \Theta^t} [\ln P(X, Z | \Theta)]$$

M 步 (Maximization, 最大化)：最大化 Q 函数，更新参数：

$$\Theta^{t+1} = \arg \max_{\Theta} Q(\Theta | \Theta^t)$$

两步交替，直到参数变化足够小。

要点：

1. EM 得到的是对数似然的**局部极大**，不一定全局最优，常需多组初始值多跑几次；
2. 思想接近坐标上升；
3. 典型应用：高斯混合模型（GMM）、有缺失时的贝叶斯网参数学习、聚类。

和 MLE 的差别：MLE 假定变量都能观测到；EM 是「有隐变量时的极大似然」，用迭代来做。

第 7 章小结

贝叶斯：先验 $\times$ 似然 $\rightarrow$ 后验。

0-1 损失下，最优就是选后验最大的类。

朴素贝叶斯：特征独立假设让计算可行，零概率用拉普拉斯平滑。

半朴素：允许每个属性再依赖一个父属性（SPODE / AODE / TAN）。

EM：隐变量问题的标准工具。

第 8 章 集成学习

集成学习：构造多个个体学习器，再用某种策略把它们的输出结合起来共同预测。通常比单个学习器更好。三个关键词：

- 个体学习器要有一定准确性（比乱猜强）；
- 个体之间要**多样**（别犯完全一样的错）；
- 结合方式要合理（投票、平均、再学习）。

「好而不同」是集成的核心。

8.1 个体与集成

结构可以看成：一组**个体学习器**各自给出结果，再经过一个**结合模块**融合成最终预测（分类或回归）。

- **同质集成**：所有个体是同一种算法（例如全部是决策树）。这时个体叫**基学习器**，所用算法叫基学习算法。
- **异质集成**：个体来自不同算法（决策树、SVM、神经网络同时出现）。这时个体叫**组件学习器**。

弱学习器：性能只比随机猜好一点。集成学习的理论结果是：可以把弱学习器提升成强学习器。

集成为什么往往更好，常见三个角度：

1. **统计**：假设空间很大，多个模型一起用，能降低「碰巧选到差模型」的风险；
2. **计算**：单个学习器容易陷入局部极小，多个起点/路径可以分担这个风险；
3. **表示**：组合之后相当于扩大了假设空间，能拟合更复杂的真实规律。

「好而不同」的定量说法是**误差-分歧分解**：

$$E = \bar{E} - \bar{A}$$

- E ：集成的泛化误差；
- $\bar{E}$ ：个体的平均误差；
- $\bar{A}$ ：分歧（多样性）。

在个体平均误差差不多时，**分歧越大，集成误差越小**。所以多样性不是点缀，而是集成能生效的关键。

现实中个体的误差不可能完全独立，所以不能靠无限堆模型一直提升。两大流派：

- **Boosting**：个体之间有强依赖，必须**串行**一个个生成；
- **Bagging / 随机森林**：个体没有强依赖，可以**并行**训练。

8.2 Boosting

基本思想：串行训练基学习器。每一轮根据上一轮的表现**调整样本权重**——分错的样本权重提高，让下一个学习器更盯这些难样本。训完 T 个基学习器后，再**加权结合**得到最终结果。

代表：**AdaBoost**

- 一开始每个样本权重相同；
- 某轮分类器训完后，分错的提高权重，分对的降低；
- 下一轮更关注难样本；
- 最终按各分类器的准确程度加权投票。

个体通常是弱学习器（如很浅的树）。Boosting 就是把弱学习器提升为强学习器。

后面的 GBDT、XGBoost、LightGBM 都是这一家族在「用梯度拟合残差」上的发展，竞赛和表格数据里极其常用。西瓜书对 GBDT 只是点到为止：它属于 Boosting，用损失函数的负梯度去拟合残差。

Boosting 特点

- 主要**降低偏差**，对弱学习器的提升很强；
- 串行训练，很难大规模并行；
- 对噪声比较敏感：错分（可能是标错）的样本权重会被不断放大，容易过拟合。

8.3 Bagging 与随机森林

Bagging

思想：用**自助法 (bootstrap)** 并行生成 T 个训练集，每个采样集训一个基学习器。

- 每个训练集都是对原数据**有放回抽样**，大小通常仍为 m ；
- 分类：简单投票；回归：简单平均。

特点：

- 可以**并行**，效率高；
- 主要**降低方差**，适合高方差、不稳定的基学习器（例如决策树）；
- 对本来就挺稳的模型（如线性模型、SVM）提升有限。

随机森林 (Random Forest)

在 Bagging「样本随机」之上，再加一层**属性随机**，进一步拉开个体差异。

两步随机：

1. **样本随机**：bootstrap 得到不同训练子集（和 Bagging 一样）；
2. **特征随机**：决策树在每个结点划分时，不看全部属性，而是先随机抽出一小撮候选属性，再从中选最优划分。

优点：双重随机带来更强多样性，泛化通常优于普通 Bagging；高度可并行；能处理高维，还能给出特征重要性。

缺点：噪声很大的回归任务上容易过拟合。

对照：Boosting 侧重降偏差；Bagging / 随机森林侧重降方差。深度模型火了之后，结构化表格数据上随机森林和 Boosting 树依然很强。

8.4 结合策略

把多个体的输出合成一个答案，常见三种：

- **平均法**：回归用简单平均或加权平均。
- **投票法**：分类用相对多数投票，也可加权；绝对多数要求过半。
- **学习法 (Stacking)**：再用一个**次级学习器**来学「怎么结合初级学习器的输出」。次级训练集要用交叉验证的方式产生，避免过拟合。

8.5 多样性

个体太像，集成没用。多样性可以用两个学习器在样本上「谁对谁错」的列联表来度量，常见名字：不合度量、Q 统计量、相关系数、 κ -统计量。入门记住「在测差异程度」即可。

增强多样性的四条路：

1. **数据样本扰动**：对原始数据采样。Bagging / 随机森林用自助采样。适合不稳定基学习器（决策树）；对 SVM、线性模型这类稳定模型效果差。
2. **输入属性扰动**：随机选部分特征来训个体。随机森林结点上的随机选属性就属于这类。适合高维数据。
3. **输出表示扰动**：改输出标签的表示，例如 ECOC（纠错输出码）。
4. **算法参数扰动**：同一算法换不同超参数，得到不同个体。

和 8.1 的误差-分歧分解一致：个体不太差的前提下，分歧越大，集成越好。

第 8 章小结

	Boosting	Bagging / 随机森林
结构	串行, 关注错分 / 残差	并行, 关注抽样多样性
偏差方差	主要降偏差	主要降方差
典型	AdaBoost、GBDT	随机森林

术语：同质 / 异质集成、基学习器、弱学习器、误差-分歧分解、自助采样、Stacking。

第 9 章 聚类

聚类是无监督学习：没有标记 y ，把相似样本放到同一簇，不相似的分开。

用途：客户分群、图像压缩、找异常点（落在稀疏区）、先探索数据再标注。

和分类的关键差别：分类有标准答案；聚类通常**没有唯一正确答案**。同一批西瓜按「甜度」或按「产地」会聚出不同结果，所以必须先想清楚「什么叫相似」。

西瓜书把聚类算法分成三类：

- **原型聚类：**用几个中心点代表簇（k-means、LVQ、GMM）。
- **密度聚类：**密的连成一片，稀的当噪声（DBSCAN）。
- **层次聚类：**一棵树，从下往上合并或从上往下分裂（AGNES）。

9.1 聚类任务

给定样本集 $D = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ ，聚类要把它分成 k 个不相交的簇 $\{C_1, \dots, C_k\}$ ，满足 $C_i \cap C_j = \emptyset \ (i \neq j)$ 且 $\bigcup_{i=1}^k C_i = D$ 。

- **硬聚类：**每个样本只属于一个簇。
- **软聚类：**每个样本以概率属于多个簇（如 GMM）。

簇 (cluster)：聚在一起的那一堆样本。

簇标记：算法给每个样本编的「第几簇」，和分类任务里事先给定的 y 不是一回事。

9.2 性能度量

外部指标（有参考标记时）

把算法得到的簇划分 C 和参考划分 C^* 对比。看两两样本的关系：

记号	含义
SS	同簇，参考里也同类
SD	同簇，参考里不同类
DS	不同簇，参考里同类

记号	含义
DD	不同簇，参考里也不同意

四个数加起来等于样本对的总数 $m(m-1)/2$ 。

Jaccard 系数：

$$JC = \frac{SS}{SS + SD + DS}$$

含义：在「至少有一方认为它们该在一起」的样本对里，两边都同意的比例。越接近 1 越好。

FM 指数 (Fowlkes-Mallows)：

$$FMI = \sqrt{\frac{SS}{SS + SD} \cdot \frac{SS}{SS + DS}}$$

Rand 指数：

$$RI = \frac{SS + DD}{SS + SD + DS + DD}$$

三者都在 $[0, 1]$ ，越大越好，都是在数「成对关系对不对」。

内部指标（没有参考时）

只看结果自身：簇内尽量近、簇间尽量远。

DB 指数 (Davies-Bouldin) 越小越好：簇内平均距离大、簇间中心距离小时，DB 会变差。

Dunn 指数 越大越好：最近的两簇分得远、最大的簇内直径小。

零基础记住一句话：**评价聚类 = 看「内部够紧、彼此够远」，有标记时再额外对一下参考答案。**

9.3 距离计算

聚类高度依赖「什么叫近」。换一种距离，簇可能完全变掉。

闵可夫斯基距离：

$$\text{dist}_p(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}|^p \right)^{1/p}$$

- x_{iu} ：样本 i 的第 u 维。
- $p = 2$ ：**欧氏距离**（直线距离，最常用）。
- $p = 1$ ：**曼哈顿距离**（只能沿网格走的街区距离）。
- $p \rightarrow \infty$ ：**切比雪夫距离**， $\max_u |x_{iu} - x_{ju}|$ （各维差值里最大的那个）。

距离度量通常要满足：非负、自己到自己为 0、对称、三角不等式。不满足三角不等式的叫「非度量距离」，有时仍能用，但不能当几何上的真距离。

其他常用「像不像」的尺子：

名称	在看什么	典型场景
余弦相似度	两个向量夹角，不看长短	文本、推荐

名称	在看什么	典型场景
相关系数	二维变化是否一起涨跌	基因表达等
VDM	离散取值在各类别下的分布差	色泽、根蒂这类属性

连续 + 离散混在一起时，要先规定怎么加权拼成一个距离，否则「色泽差一档」和「含糖率差 0.1」无法比较。

9.4 原型聚类

用一组**原型 (prototype)** 代表簇，样本归到最近的原型。

k 均值 (k-means)

目标：最小化各点到其簇中心的平方距离之和 (SSE)：

$$E = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|_2^2$$

- C_i : 第 i 簇。
- $\boldsymbol{\mu}_i$: 第 i 簇的均值向量 (中心) , $\boldsymbol{\mu}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$ 。
- E 越小：点越抱紧自己的中心。

算法步骤：

1. 随机选 k 个中心。
2. **划分**：每个点归到最近中心。
3. **更新**：用每簇样本的均值当新中心。
4. 重复 2 ~ 3，直到中心几乎不动 (或 E 几乎不再下降)。

优点：简单、快。

缺点：要预先定 k ；对初值敏感；默认球形、差不多大的簇；噪声和离群点会把中心拽偏。

实践：多次随机初始化取 E 最小的；或用 k-means++ 让初始中心尽量散开。 k 可用肘部法 (看 E 随 k 下降是否变缓) 或轮廓系数辅助选择。

学习向量量化 LVQ

原型学习，但有**标记**。每个原型 $\mathbf{p}$ 带着类别。

遇到一个样本 $(\mathbf{x}, y)$ ，找到最近原型 $\mathbf{p}_{i^*}$ ：

- 若 y 与 $\mathbf{p}_{i^*}$ 同类： $\mathbf{p}_{i^*} \leftarrow \mathbf{p}_{i^*} + \eta(\mathbf{x} - \mathbf{p}_{i^*})$ (往样本拉近)；
- 若异类： $\mathbf{p}_{i^*} \leftarrow \mathbf{p}_{i^*} - \eta(\mathbf{x} - \mathbf{p}_{i^*})$ (推远)。

η 是学习率。像「带监督的 k-means」，学出来的原型可当该类代表。

高斯混合聚类 GMM

假设数据由 k 个高斯分布按比例混合而成：

$$p(\mathbf{x}) = \sum_{i=1}^k \alpha_i \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$$

- α_i ：第 i 个高斯的混合系数， $\alpha_i \geq 0$ 且 $\sum_{i=1}^k \alpha_i = 1$ 。
- $\boldsymbol{\mu}_i$ ：第 i 个高斯的中心。
- $\boldsymbol{\Sigma}_i$ ：协方差，决定簇是圆的还是扁的、朝哪斜。
- $\mathcal{N}(\cdot)$ ：高斯（正态）密度，中心高、四周低。

用 **EM 算法** 学这些参数。样本 $\mathbf{x}_j$ 由第 i 个高斯生成的后验（责任度）：

$$\gamma_{ji} = \frac{\alpha_i \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{l=1}^k \alpha_l \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}$$

- **E 步**：按当前参数算每个点的 γ_{ji} 。
- **M 步**：用 γ_{ji} 当权重，更新 $\alpha, \boldsymbol{\mu}, \boldsymbol{\Sigma}$ 。

GMM 是**软聚类**：一个点可以 0.7 属于簇 A、0.3 属于簇 B。k-means 可看成 GMM 在「协方差相同且很小、归属非 0 即 1」时的特例。GMM 能拟合椭圆簇，比 k-means 灵活，但也更慢、更依赖初值。

9.5 密度聚类

代表 **DBSCAN**：密集处连成一簇，稀疏处当噪声。不必先指定球形，能发现「月牙形」等任意形状。

样本 $\mathbf{x}$ 的 ε -邻域：

$$N_\varepsilon(\mathbf{x}) = \{\mathbf{x}' \in D \mid \text{dist}(\mathbf{x}, \mathbf{x}') \leq \varepsilon\}$$

两个参数：

- ε ：邻域半径；
- MinPts：邻域里至少多少个点，才算**核心对象**，即 $|N_\varepsilon(\mathbf{x})| \geq \text{MinPts}$ 。

几个术语：

- **核心对象**： ε 邻域内点够多，自己是「密区里的人」。
- **密度直达**：在某个核心对象邻域内的点，可由它直达。
- **密度可达**：一串核心对象接力过去（「直达」的传递）。
- **密度相连**：共同从某个核心对象可达。
- **噪声 / 离群点**：谁的密区都进不去的点。

算法直觉：从任意还未处理的核心对象出发，把所有密度可达的点收进同一簇，再换下一个核心对象。

优点：任意形状、能标噪声、不必预先定簇数。

缺点： ε 和 MinPts 很敏感；密度不均匀时，一组参数很难同时适合所有簇。

9.6 层次聚类

代表 **AGNES**（自底向上）：

1. 每个样本先自成一簇。
2. 每次合并距离最近的两簇。
3. 直到只剩指定的 k 簇，或全部并成一棵树。

结果是**树状图 (dendrogram)**，在不同高度切开就得到不同粗细的分群。

两簇 C_i, C_j 的距离怎么定义，会改变树的形状：

链接方式	公式直觉	特点
单链接（最小距离）	两簇里最近的一对点	易出现「链条」，把细长结构连起来
全链接（最大距离）	两簇里最远的一对点	簇更紧、更圆
平均距离	两两距离取平均	较折中
中心距离	$\ \mu_i - \mu_j\ $	接近原型聚类的味道

DIANA 是反过来：先全部当一簇，再不断分裂。了解有「自顶向下」这条路即可。

第 9 章小结

方法	类型	记住
k-means	原型、硬聚类	快，球形簇，要定 k
LVQ	原型、有监督	同类拉近、异类推远
GMM	原型、软聚类	EM + 责任度，椭球簇
DBSCAN	密度	任意形状，能标噪声
AGNES	层次	树状合并，切不同高度得不同 k

术语：簇、原型、硬/软聚类、责任度、核心对象、密度可达、树状图。

第 10 章 降维与度量学习

维数 d 很大时会出现**维数灾难**：

- 空间极空，样本再多也显得稀；
- 「远近」变得差不多，kNN 这类靠距离的方法失效；
- 需要的样本数随维数指数增长。

降维：把 $\boldsymbol{x} \in \mathbb{R}^d$ 变成更短的 $\boldsymbol{z} \in \mathbb{R}^{d'}$ ($d' \ll d$)，同时尽量保住有用结构（距离、近邻、方差、可分性）。

本章先讲一个极度依赖距离的方法 **kNN**，再讲怎么找低维表示：MDS、PCA、核 PCA、流形学习，最后是**学一种更合适的距离**（度量学习）。

10.1 k 近邻 kNN

kNN (k-Nearest Neighbors) 属于监督学习，也是**惰性学习 (lazy learning)**：训练阶段几乎不做事，只把全部训练样本存下来，**预测时才算**。相对地，决策树、SVM、神经网络是**急切学习**，训练时就把模型定下来。

它还是**无参数、基于实例**的：不显式学一组 w ，答案直接来自具体邻居。

预测规则：在训练集里找离测试样本最近的 k 个邻居。

- **分类**：多数投票；也可以按距离加权，近的权重大。
- **回归**：邻居输出取平均或加权平均。

$$f(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \mathbb{I}(y_i = c)$$

$k = 1$ 就是最近邻。 k 太小对噪声敏感（方差大）； k 太大掺进远的、甚至别类的点（偏差大）。二分类常试奇数 k ，用验证集选。

理论：在**密采样假设**下（样本无穷多、处处都够密），最近邻分类器的错误率不超过贝叶斯最优分类器的两倍。现实样本有限，这个条件很难满足；**高维下距离失效，kNN 会急剧变差**，正是维数灾难的典型表现。所以后面要降维或换度量。

10.2 低维嵌入与 MDS

很多数据看起来在高维，实际贴在一个低维曲面（**流形**）上。例如：一张人脸照片有成千上万像素，但真正变化的只有表情、光照、姿态等少数因素。

低维嵌入：找一组低维坐标，让高维里的近邻、距离关系尽量还在。

MDS（多维缩放）

输入是原始空间里样本两两的距离矩阵。目标：在低维里重构这些点，**两两距离尽量和原来一致**。

$$\min_{z_1, \dots, z_m} \sum_{i < j} (\|z_i - z_j\| - \text{dist}_{ij})^2$$

做法是由距离矩阵还原内积矩阵，再做特征值分解，得到低维坐标。若原始距离就是欧氏距离，MDS 与 PCA 在旋转意义下等价。

10.3 主成分分析 PCA

最常用的线性降维。两个**等价**的优化目标：

1. **最近重构性**：投影到低维后再还原回高维，**重构误差最小**。
2. **最大可分性**：投影后方差尽可能大，样本尽量散开，留下最多信息。

先把数据中心化（每个属性减均值，把数据平移到原点），否则第一成分会被「整体平移」抢走。协方差矩阵

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$$

（也可写成中心化后的 $XX^\top$ 一类形式。）第一主成分是最大特征值对应的单位特征向量：

$$\max_w \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{s.t.} \quad \mathbf{w}^\top \mathbf{w} = 1$$

执行步骤：

1. 数据中心化；
2. 算协方差矩阵，做**特征值分解**，特征值从大到小排序；
3. 取前 d' 个最大特征值对应的特征向量，组成投影矩阵 W ；
4. 投影： $\mathbf{z}_i = W^\top (\mathbf{x}_i - \bar{\mathbf{x}})$ 。

重构回原空间： $\hat{\mathbf{x}} = \bar{\mathbf{x}} + W\mathbf{z}$ 。

d' 怎么选：可以人工指定；也可以按**累计方差百分比**，例如保留 95% 方差，取满足

$$\frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{i=1}^d \lambda_i} \geq 0.95$$

的最小 d' 。

注意：

- PCA **不管类别标签**，是无监督的。某方向方差很大，未必对分类有用。想「对分类最有用」的线性投影，用第 3 章的 **LDA**。
- 特征尺度差很多时要先标准化，否则数值大的维会主导主成分。
- 优点：简单、快、几乎没有超参数。
- 缺点：**只能捕捉线性相关**，对非线性流形差；对异常点敏感。

深度学习里常用 PCA 做可视化、加速预处理，或理解协方差结构。

10.4 核化线性降维 KPCA

PCA 是线性的，弯成香蕉形的数据投影后会叠在一起。**KPCA** 用核技巧：先隐式映射到高维，再在那里做 PCA，不必写出 $\phi(\mathbf{x})$ 。

构造核矩阵 $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ ，替代原来的内积矩阵，做中心化后的特征分解，取前 d' 个特征向量得到投影。

本质：把线性 PCA 里的内积全部换成核函数，和 SVM 是同一招。

优点：能处理非线性结构。

缺点：核矩阵是 $m \times m$ ，样本一多又慢又占内存。

10.5 流形学习

假设数据在低维流形上，重点**保持局部几何**，而不是全局直线距离。

等度量映射 Isomap

流形上的真实距离不是直线欧氏距离，而是**测地线距离**（沿着曲面表面走的最短路径）。高维直线会「抄近道穿过空洞」。

步骤：

1. 对每个样本找 k 近邻，建近邻图，近邻之间用欧氏距离；

2. 用 **Floyd (或 Dijkstra)** 算图上两点最短路径，近似测地距离，得到距离矩阵；
3. 把这个距离矩阵交给 MDS，得到低维坐标。

适合「卷着的瑞士卷」那种数据。

局部线性嵌入 LLE

核心假设：**流形上的一小块近似是线性的**。

1. 每个样本找 k 个近邻，用它们的线性组合重构自己，得到权重 w_{ij} ($\sum_j w_{ij} = 1$)：

$$\min_W \sum_{i=1}^m \left\| \mathbf{x}_i - \sum_{j \in N_k(\mathbf{x}_i)} w_{ij} \mathbf{x}_j \right\|^2$$

2. 权重固定，求低维坐标，使低维里仍满足同一组局部线性关系：

$$\min_{\mathbf{z}_1, \dots, \mathbf{z}_m} \sum_{i=1}^m \left\| \mathbf{z}_i - \sum_j w_{ij} \mathbf{z}_j \right\|^2$$

3. 对一个稀疏矩阵做特征分解，得到低维嵌入。

优点：只保**局部邻域结构**，不必算全局距离。

缺点：对 k 敏感；流形有孔洞、断裂时效果差。后来可视化常用的 t-SNE、UMAP，思想同属「保局部结构」。

对照：**Isomap 保全局测地距离；LLE 保局部线性重构关系**。

10.6 度量学习

欧氏距离默认「每一维同样重要、正交」。这常常不对：判断好瓜时，「含糖率差 0.01」和「编号差 1」不该同等对待。

降维是变换样本坐标；**度量学习不改样本本身，直接学一个距离矩阵 M** 。

常见形式是马氏距离：

$$\text{dist}_M(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^\top M (\mathbf{x}_i - \mathbf{x}_j)}$$

- M 必须对称半正定 ($M \succeq 0$)，距离才非负、对称。
- $M = I$ 时退回欧氏距离。
- 等价于学一个线性变换 L ($M = L^\top L$)，在变换后的空间里用欧氏距离。

目标：利用训练数据学最优 M ，让**同类尽量近、异类尽量远**。代表算法 **LMNN (大间隔最近邻)**：专门给 kNN 学一把更好的尺子。

人脸验证、图像检索都常先学度量。深度学习里则直接学嵌入 (triplet loss、对比学习)，再用余弦或欧氏比较——是度量学习的深度版本。

第 10 章小结

方法	在保什么	监督?
kNN	不降维，直接用邻居	用标记投票

方法	在保什么	监督?
MDS	两两距离	通常无监督
PCA	全局方差 / 重构	无监督
LDA	类间远、类内近	监督
KPCA	核空间里的方差	无监督
Isomap	流形上的测地距离	无监督
LLE	局部线性重构权重	无监督
度量学习 / LMNN	学「该怎么量远近」	通常监督

术语：维数灾难、惰性学习、密采样、低维嵌入、最近重构 / 最大可分、方差解释比例、测地距离、马氏距离、LMNN。

第 11 章 特征选择与稀疏学习

不是特征越多越好。无关特征、冗余特征会：加重过拟合、让训练变慢、让模型难解释。

两条路，目标接近：

- **特征选择：**从已有 d 个特征里挑一个子集，剔除冗余、无关特征。
- **稀疏学习：**训练时让很多权重变成 0，用少量非零参数 / 特征表达模型，等于自动关掉一些特征。

特征选择 ≠ 降维：

- 降维（如 PCA）：生成**新的复合特征**，一般是原始特征的组合，往往失去原来的物理含义。
- 特征选择：直接从原始特征里**挑一部分留下来**，含义还在（例如「判断好瓜真正有用的是含糖率和根蒂」）。

11.1 子集搜索与评价

特征选择分两步：**子集搜索** + **子集评价**。

全部特征子集有 2^d 个， d 一大穷举完全不可行（NP 难），只能启发式、贪心搜索，不保证全局最优。

子集搜索

- **前向搜索：**从空集出发，每次加入当前评价最好的一个特征。
- **后向搜索：**从全集出发，每次删掉最不伤评价的一个。
- **双向搜索：**既能加也能减，减轻「选错了不能悔」。

子集评价

对搜到的子集打分。常用信息熵、信息增益：给定特征子集，看数据被划分后的条件熵；**信息增益越大，这个子集越好。**

按「评价时要不要用到后续那个学习器」，分成三类：**过滤式、包裹式、嵌入式。**

11.2 过滤式选择 Filter

先按统计指标给每个特征（或特征子集）打分，**和后续学习器无关**；筛完再把子集交给模型。

常见打分：与标记的**相关系数**、互信息、信息增益，以及 Relief。

Relief

用**近邻**给每个特征算统计权重，看它能不能把不同类分开。

1. 随机抽一个样本 x 。
2. 找两个邻居：
 - **猜中近邻 (near-hit)**：和 x 同类的最近样本；
 - **猜错近邻 (near-miss)**：和 x 不同类的最近样本。
3. 更新第 j 维权重：该维上「离同类近、离异类远」就加分；反过来（同类差很大、异类却差不多）就减分。

$$\delta_j^j \leftarrow \delta_j^j - \text{diff}(x_j, x_j^{\text{nh}})^2 + \text{diff}(x_j, x_j^{\text{nm}})^2$$

算出全部特征权重后，设一个阈值，留下权重大于阈值的特征。二分类用 Relief；**Relief-F** 拓展到多分类。

优点：计算快，不依赖学习器，能当预处理。

缺点：主要看特征单独的表现，容易忽略「几个特征合在一起才有用」，也容易漏掉冗余。

11.3 包裹式选择 Wrapper

直接把后续学习器的性能，当作特征子集的评价标准。 用它在某个子集上的交叉验证结果来搜索。

可以理解为「用模型当黑盒评分器来挑特征」。

书里的 **LVW (拉斯维加斯包装算法)**：随机生成特征子集，在训练集上训练学习器，用交叉验证评估；若这个子集更好，就留下来，直到时限用完。

优点：直接面向最终任务，会考虑特征**组合**效果，通常比 Filter 准。

缺点：每个候选子集都要完整训一遍模型，**计算开销巨大**；也更容易过拟合。

对照：Filter 是「先选特征，再训练模型」；Wrapper 是「用模型结果评判特征好坏」。

11.4 嵌入式选择与 L1 正则化

学习和选特征**同时**进行。决策树每次选划分属性，本身就是嵌入式选择。训练一次同时得到模型 + 特征子集，开销远小于 Wrapper。

线性模型上最经典的是正则化。在损失后面加一项，限制权重别太大：

$$\min_w \frac{1}{m} \sum_{i=1}^m \ell(y_i, \mathbf{w}^\top \mathbf{x}_i + b) + \lambda \Omega(\mathbf{w})$$

- ℓ ：拟合数据的损失（平方误差、交叉熵等）。
- $\Omega(\mathbf{w})$ ：正则项，描述「模型有多复杂」。
- λ ：越大，越强调简单、越稀疏、越不容易过拟合。**调 λ 很关键。**

三种常见的 Ω :

	L0	L1 (Lasso)	L2 (岭回归)
形式	非零权重的个数	$\ \mathbf{w}\ _1 = \sum_j w_j $	$\ \mathbf{w}\ _2^2 = \sum_j w_j^2$
效果	直接控制用了几个特征	部分权重变成 精确的 0	权重整体变小，很少精确为 0
备注	组合爆炸，很少直接优化	凸的，可当特征选择	深度学习里叫权重衰减

几何直觉: L2 的约束像圆, L1 的约束像菱形; 菱形的尖角在坐标轴上, 最优点更容易碰到轴, 于是某些 $w_j = 0$ 。L1 常被看成 L0 的凸替代。

L1+L2 一起用叫 Elastic Net, 既稀疏又比较稳 (特征高度相关时 Lasso 会随便丢掉其中一个, Elastic Net 更温和)。

11.5 稀疏表示与字典学习

普通线性模型是「特征 $\times$ 权重」。稀疏表示反过来: 准备一本**字典** D (很多原子向量当列), 每个样本用**很少几个原子的线性组合**来写:

$$\min_{\alpha} \|\mathbf{x} - D\alpha\|_2^2 + \lambda \|\alpha\|_1$$

- D : 字典矩阵, 每一列是一个原子。
- α : 稀疏系数, L1 项逼它大部分为 0。

像造句只用很少几个单词。字典可以预先给定 (如傅里叶变换基), 也可以从数据里和系数一起学:

$$\min_{D, A} \|X - DA\|_F^2 + \lambda \|A\|_1$$

这叫**字典学习**。用于去噪、压缩、识别。深度学习的卷积核、稀疏编码层, 有这一思想的影子。

11.6 压缩感知

若信号 $\mathbf{x}$ 本身稀疏 (或在某组基下稀疏), 可以先做少量线性测量 $\mathbf{y} = \Phi\mathbf{x}$, 再通过带 L1 的优化把 $\mathbf{x}$ 恢复出来。采样次数可以**远少于**传统奈奎斯特定理所要求的。

关键条件 (知道名词即可):

- 信号本身稀疏 (或在某变换下稀疏);
- 测量矩阵 Φ 和稀疏基**不相干** (测量要「搅得比较均匀」)。

医学 MRI 加速成像是著名应用。对入门: 它说明「稀疏不只是省参数, 还能少采数据」。

第 11 章小结

类型	怎么做	特点
Filter	先打分再交给模型 (相关、Relief)	快, 不管后续模型
Wrapper	用模型交叉验证性能搜索子集 (如 LVW)	准但极慢

类型	怎么做	特点
Embedded	学习过程中顺便选，如 L1	一举两得，开销小于 Wrapper

术语：无关 / 冗余特征、前向后向搜索、Relief / Relief-F、LVW、L0/L1/L2、Lasso、字典学习、压缩感知。

第 12 章 计算学习理论

前面各章都是「这个算法怎么用」。这一章问更根本的问题：

- 怎样才算「学得会」？
- 大概需要多少样本？
- 模型多复杂时，训练误差小也不保证泛化好？

零基础不要卡在证明。抓住 PAC、VC 维、复杂度惩罚这几个词，就能看懂很多论文里「泛化界」在说什么。

12.1 基础知识

把学习看成：从假设空间 $\mathcal{H}$ 里挑一个假设 h ，希望它的**泛化误差**（在未知分布上的对错比例）小。

经验误差与泛化误差

- **经验误差** $\hat{\epsilon}(h)$ ：在训练样本集上的错误率。
- **泛化误差** $\epsilon(h)$ ：在真实分布上的错误率。

我们希望 $\epsilon(h)$ 尽量小，但只能直接看到 $\hat{\epsilon}(h)$ 。样本有限时两者会有缺口。学习理论要给这个缺口一个上界：缺口通常随「模型越复杂」变大，随「样本越多」变小。

常用符号

符号	含义
$\mathcal{X}$	样本空间
$\mathcal{Y}$	标记空间
$\mathcal{D}$	样本的真实分布
$\mathcal{H}$	假设空间（所有可能假设的集合）
m	训练样本个数

可实现与不可实现

- **可实现（可分）**：真实目标概念 $c \in \mathcal{H}$ ，最优假设就在假设空间里。
- **不可实现**：真实概念不在假设空间中。

12.2 PAC 学习

PAC 回答：需要多少样本，才能以高概率得到一个「差不多正确」的假设。

PAC (Probably Approximately Correct, 概率近似正确)：不要求永远正确、也不要求误差精确为 0，只要求同时满足：

- **Probably**：以很大的概率 $1 - \delta$ （例如 95% 的随机训练集都能成功）；
- **Approximately**：学到的模型泛化误差不超过 ε （例如最多差 1%）。

ε 叫精度 / 误差参数， δ 叫置信参数（失败概率）。两者都是事先指定的小数。

若存在算法 $\mathcal{L}$ ，当样本数 m 足够大时输出假设 h ，满足

$$P(\epsilon(h) \leq \varepsilon) \geq 1 - \delta$$

就说：这个假设的泛化误差不超过 ε 的概率至少为 $1 - \delta$ 。

PAC 可学习

若存在多项式时间的算法，且只需

$$m = \text{poly}\left(\frac{1}{\varepsilon}, \frac{1}{\delta}\right)$$

个样本就能做到上面这件事，则称这个概念类是 **PAC 可学习** 的。

直观：想要更准（ ε 更小）或更有把握（ δ 更小），通常需要更多样本；假设空间越大，同样保证下需要的样本也更多。

样本复杂度

达到 PAC 条件所需要的最少样本数。

可实现情形（目标概念属于 $\mathcal{H}$ ）：

$$m \geq \frac{1}{\varepsilon} \left(\ln |\mathcal{H}| + \ln \frac{1}{\delta} \right)$$

$|\mathcal{H}|$ 越大，需要的样本越多。经验误差最小的那个假设，其泛化误差上界和 $\log |\mathcal{H}|$ 成正比。

不可实现情形（真实概念不在 $\mathcal{H}$ 里）：仍存在贝叶斯最优一类的参照，但样本复杂度通常更高。这时仍能以大概率保证：你选的假设不会比 $\mathcal{H}$ 里最好的差太多。

PAC 的局限：原始形式主要适合**离散有限假设空间**。SVM、神经网络这类实数参数使 $|\mathcal{H}| = \infty$ ，不能再 $|\mathcal{H}|$ 计数，于是引出 VC 维。

12.3 有限假设空间

若 $\mathcal{H}$ 里只有有限个假设（例如所有「深度不超过 3、属性取值离散」的决策树），上一节的样本复杂度和下面的泛化界都可以用 $|\mathcal{H}|$ 写出来。

$$\text{泛化误差} \lesssim \text{训练误差} + \sqrt{\frac{\ln |\mathcal{H}| + \ln(1/\delta)}{m}}$$

读法：**训练误差 + 一项随假设个数变大、随样本数 m 变小的惩罚**。这就是「为什么假设空间不能乱放」的定量版。

可分（可实现）时：样本够多，以大概率能找到泛化误差很小的假设。

不可分（不可实现）时：永远可能有一点误差，但仍能以大概率保证你选的那个不会比 $\mathcal{H}$ 里最好的差太多。

12.4 VC 维

现实中 $\mathcal{H}$ 往往无穷（所有可能的直线、所有可能的神经网络权重）。不能再用 $|\mathcal{H}|$ ，要换一种「复杂度」。

打散 (shatter)：对一组样本，你能用 $\mathcal{H}$ 里的假设实现它们的**任意一种**正负标记。

VC 维： $\mathcal{H}$ 能打散的最大样本数。它刻画无限假设空间的表达能力，**和样本分布无关**。

例子：平面上的线性分类器，VC 维是 3——3 个不共线的点，8 种标法都能用直线实现；4 个点就存在一种标法（对角两类）直线划不开。

含义：

- VC 维越高：模型族越灵活，越容易把训练集拟合到近乎 0 误差，也更容易过拟合；
- 同样的泛化保证下，VC 维高往往意味着需要更多数据。

神经网络的 VC 维可以非常高，所以经典 VC 界对深度网络常常**太松**，不能直接拿来算「要多少样本」。但定性结论仍对：**能力太强 + 数据太少 = 泛化堪忧**，要用正则、结构约束、更多数据。

12.5 Rademacher 复杂度

另一种复杂度：给样本随机贴上 ± 1 的噪声标签，看 $\mathcal{H}$ 能把这些噪声拟合到什么程度。

能把随机噪声都拟合得很好 $\rightarrow$ 假设空间太强 $\rightarrow$ Rademacher 复杂度高 $\rightarrow$ 泛化界更松。

和 VC 维的差别：VC 维不考虑数据分布；Rademacher 复杂度**依赖数据分布**，因此往往更精细、界更紧。由 VC 维可以推出 Rademacher 的上界。

Rademacher 随机变量

$\sigma_i \in \{+1, -1\}$ ，每个以 $1/2$ 的概率取 $+1$ 或 -1 。

经验 Rademacher 复杂度：

$$\hat{\mathfrak{R}}_m(\mathcal{H}) = \mathbb{E}_{\sigma} \left[\sup_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma_i h(\mathbf{x}_i) \right]$$

直观：给样本随机翻转标签，看假设空间最多能把这套随机噪声拟合到多好。拟合得越好，说明表达能力越强，越容易过拟合。

基于它可以给出经验误差与泛化误差之间的界：Rademacher 复杂度越大，过拟合风险越高。

12.6 稳定性

算法稳定性：训练集换掉一个样本，学出的模型别抖太厉害。

VC 维、Rademacher 复杂度是**假设空间的性质**；稳定性直接描述**学习算法本身**。即使假设空间很大，只要算法足够稳定，仍可能有泛化保证。

均匀稳定

把训练集 D 和去掉第 i 个样本后的 $D \setminus i$ 分别交给算法，得到 $L(D)$ 和 $L(D \setminus i)$ 。若两者的预测差别有上界，就称该算法**均匀稳定**。稳定算法可以推出泛化误差上界。

例子：

- 带 L2 正则的 SVM 是稳定算法；
- 未正则化的决策树不稳定（样本稍改，树结构可能大变）。

正则化、平均（Bagging）都能提高稳定性。这从「对数据别过敏」解释了第 2 章的方差。

第 12 章小结

术语	一句话
PAC	以大概率 $1 - \delta$ 、近似到 ϵ 地学对
样本复杂度	达到 PAC 至少要多少样本
$\ \mathcal{H}\ $	有限假设空间用它；越大越要更多数据
VC 维	能打散多少点；与分布无关，界较松
Rademacher	拟合随机噪声的能力；依赖分布，界更紧
稳定性	换一个样本，算法输出别乱跳

PAC 是计算学习理论的基础：有限假设空间用 $|\mathcal{H}|$ ，无限假设空间用 VC 维。

Rademacher 把数据分布也考虑进去。

稳定性不单看假设空间有多强，而看学习算法有多鲁棒。

对深度学习：经典界往往太松，但「复杂度要和数据量匹配、正则能换稳定性」这条直觉一直成立。

第 13 章 半监督学习

监督学习要大量标记，标记贵；未标记数据往往很容易收集（网上的图、没人工打标签的西瓜照片）。

半监督学习：同时用少量有标记样本和大量未标记样本来学。

未标记样本不是白捡的。它们有用，前提是数据分布有结构。常见两大假设：

- **聚类假设**（低密度分离假设）：同类样本挤在同一簇，分类边界应该穿过稀疏地带，而不是从密堆中间切开。
- **流形假设**（平滑假设）：高维点实际贴在低维流形上，流形上邻近的点标记应相似。

若假设不成立（未标记数据和有标记数据根本不是一回事），硬用未标记数据可能**有害**。

另外要分清：

- **纯半监督**：训练时用未标记数据，预测的是训练时没见过的新样本。
- **直推学习 (transductive)**：未标记样本在训练时就在，只要求把**这些**未标记样本标对，不强调对新的 x 泛化。

- **主动学习**：会主动挑选样本，请专家标注，把未标记转为标记样本，需要外部交互，**不属于**半监督学习。

13.1 未标记样本

未标记样本**没有标签，但带着整体数据分布的信息**。如果直接丢掉、只用那点有标记样本，模型很容易很差。

重要前提：**标记样本与未标记样本服从同一个数据分布（独立同分布，i.i.d.）**。分布不一致时，半监督反而会损害性能。

13.2 生成式方法

核心假设：全部样本（标记 + 未标记）来自**同一个生成模型**。例如每个类是一个高斯，未标记点也从这些高斯里抽出来。有标记样本帮助定各类参数，未标记样本帮助把形状、比例估得更准。

做法：

1. 假定数据服从某生成分布（如 GMM），参数包括混合系数、各高斯的均值和协方差。
2. 未标记样本的标签当作**隐变量**，用 **EM** 迭代：
 - **E 步**：按当前参数，估未标记样本属于每个成分的后验（伪标签）；
 - **M 步**：结合有标记 + 未标记样本，更新全部参数。
3. 收敛后，用学得生成模型做分类。

优点：理论清楚；生成假设对得上真实数据时效果好。

缺点：**假设和真实数据不符时会严重变差**；计算也更贵。可能还不如只用那点有标记数据。

13.3 半监督 SVM

普通 SVM 只管有标记点的间隔。半监督 SVM (**S3VM / TSVM**) 还希望：**分类边界不要穿过未标记数据的稠密区**，未标记点尽量远离边界（分到哪一侧由算法一起决定）。

对照：

- 普通监督 SVM：最大化间隔，分开有标记样本。
- S3VM：还要让超平面穿过**低密度空白区**，避开稠密簇。直觉就是聚类假设：边界走「空地」。

目标函数：

$$\min_{w, b, \hat{y}, \xi, \hat{\xi}} \frac{1}{2} \|w\|^2 + C_l \sum_{i=1}^l \xi_i + C_u \sum_{j=l+1}^{l+u} \hat{\xi}_j$$

- C_l ：有标记样本的惩罚系数；
- C_u ：未标记样本的惩罚系数，训练初期往往较小，再逐步增大；
- 有标记样本仍满足普通 SVM 间隔约束；未标记样本要求 $|w^\top x_j + b| \geq 1 - \hat{\xi}_j$ 。

TSVM 是组合优化，NP 难，没有闭式解。常用**局部搜索迭代**：

1. 只用有标记样本训一个初始 SVM；
2. 给未标记样本赋伪标签；

3. 迭代交换部分未标记样本的伪标签，优化目标，并逐步提高 C_u ，直到收敛。

优点：最大间隔 + 低密度假设。

缺点：NP 难，计算贵，容易陷入局部最优。

13.4 图半监督学习（标签传播）

把全部样本建成一张图：**结点是样本，边权是相似度**。有标记结点带着标签，标签沿边向邻居**传播**，未标记样本从邻近的有标记样本拿到标签。经典算法叫**标签传播（Label Propagation）**。

目标通常两项：

- **损失项**：有标记样本的预测尽量贴合真实标签；
- **平滑项**：图上相邻结点的输出尽量接近（流形 / 平滑假设）。

做法直觉：

1. 建图：标记、未标记样本都当结点；邻接权重矩阵 W 常用高斯核算两两相似度（也可用 k 近邻或 ε 邻域）。
2. 在图正则目标下求解，得到所有结点（含未标记）的输出标签。

图怎么建很关键，对结果影响很大。样本多时图矩阵很大，存储和计算都贵。这类方法多半是**直推学习**，训练时没在图上的新测试样本一般不能直接处理。

13.5 基于分歧的方法

核心：利用**多个学习器之间的分歧（多样性）**，互相挑选高置信度的伪标记样本，扩充对方的训练集。代表是**协同训练（Co-Training）**，针对**多视图数据**。

多视图：同一个对象有多组互不重叠的属性，每一组叫一个视图。例如网页：视图 1 是正文，视图 2 是超链接信息。

两个条件：

1. **相容性**：每个视图单独训练，都能得到不错的学习器；
2. **互补性**：两个视图信息不完全一样，可以互相补充。也就是书上说的「充分且条件独立」。

Co-Training 流程

1. 用有标记样本，分别在视图 1、视图 2 上各训一个基学习器。
2. 每个学习器从**未标记样本里挑自己最有把握的**，打上伪标签。
3. 把这些伪标记样本交给**另一个学习器**，扩充对方的训练集。
4. 反复迭代，直到没有合适样本可再选。

没有天然多视图时，也可以用不同算法、不同特征子集、不同扰动来制造分歧，让两个模型互相教。核心是：它们错的地方不完全一样，才能互补。

深度学习里的伪标签、师生模型、FixMatch（对未标记样本做强扰动，要求预测仍一致），都是分歧 / 一致性思想的现代版。

优点：不必对数据分布做显式概率假设。

缺点：强依赖多视图；视图不满足相容、互补时效果会变差。

13.6 半监督聚类

聚类本身是无监督的。半监督聚类是：再提供少量监督信息，辅助分簇。

两种成对约束：

- **必连 (must-link)**：这两个样本必须同一簇；
- **勿连 (cannot-link)**：这两个样本一定不能同一簇。

算法在 k-means 等流程里尊重这些约束，或把约束变成对距离的修改。这叫**约束聚类 / 半监督聚类**（如约束 k-means）。

第 13 章小结

方法	核心假设 / 做法
生成式	统一生成分布 + EM，标签当隐变量
S3VM / TSVM	最大间隔 + 边界走低密度区
图半监督	建图 + 标签传播，偏直推
Co-Training	多视图、学习器分歧、互喂伪标签
约束聚类	必连 / 勿连

1. 半监督利用**大量未标记样本的分布信息**；要分清纯半监督、直推、主动学习；前提是标记与未标记 i.i.d.。两大假设：**聚类（低密度分离）**、**流形平滑**。
2. 生成式：未标记标签当隐变量，EM 求解；生成假设错则效果差。
3. S3VM / TSVM：超平面落在低密度区；NP 难，迭代局部搜索，逐步增大 C_u 。
4. 图方法：样本建图，标签传播；直推，大图开销大。
5. Co-Training：多视图、相容互补；两个学习器互挑高置信伪标签。
6. 半监督聚类：必连、勿连，约束 k-means。

术语：半监督、直推、主动学习、聚类假设、流形假设、伪标签、S3VM / TSVM、标签传播、协同训练、must-link / cannot-link。

第 14 章 概率图模型

联合分布 $P(x_1, x_2, \dots, x_n)$ 直接写出来，参数会随变量个数爆炸。**概率图模型**用图把「谁依赖谁」画出来：结点是随机变量，边是依赖；没有边就表示条件独立，联合分布可以拆成很多局部小块相乘。

两种图：

- **有向图 / 贝叶斯网**：箭头表示直接依赖（常带点因果味道）。第 7 章的贝叶斯网属于这类。HMM 也是有向的。
- **无向图 / 马尔可夫网 / 马尔可夫随机场 (MRF)**：边只表示相关，没有方向。

两大任务：

- **学习**：从数据估图结构，或势函数 / 条件概率表的参数。

- **推断**：已知一部分观测，求未知变量的后验或边际概率。分**精确推断**和**近似推断**。

14.1 隐马尔可夫模型 HMM

用于**序列**。变量分成两排：

- **隐状态** $y_1, y_2, \dots, y_T$ ：看不见，如「这句话每个位置的词性」。
- **观测** $x_1, x_2, \dots, x_T$ ：看得见，如「实际听到的声音帧」。

两个假设：

1. **马尔可夫性**：下一状态只依赖当前状态， $P(y_{t+1} | y_t)$ ，更早的历史不管。
2. **观测独立**：当前观测只依赖同一时刻的隐状态， $P(x_t | y_t)$ 。

三组核心参数 $\lambda = (\pi, A, B)$ ：

- π ：初始状态概率；
- A ：状态转移矩阵， $a_{ij} = P(y_{t+1} = s_j | y_t = s_i)$ ；
- B ：发射（观测）概率矩阵， $b_{ik} = P(x_t = o_k | y_t = s_i)$ 。

三大经典问题：

问题	问什么	典型算法
评估	已知 λ ，观测序列概率 $P(x_1 \dots x_T \lambda)$ 多大	前向 / 后向
解码	最可能的隐状态序列是什么	维特比 (Viterbi, 动态规划)
学习	只有观测，怎么估 λ	Baum-Welch / 前向-后向 (EM)

语音识别、中文分词的经典基线都用过 HMM。

14.2 马尔可夫随机场 MRF（无向图 / 马尔可夫网）

无向图，边没有方向，刻画变量之间的**对称依赖**。不能像贝叶斯网那样直接写成一串条件概率，而是靠**势函数**建模。

一个变量的条件分布只依赖于它的**邻居**。更大一点：给定某个结点的邻居（**马尔可夫毯**），它与图上其余结点独立。

团与极大团

- **团**：结点子集，内部任意两点之间都有边。
- **极大团**：再加任何一个结点就不再是团。

联合概率分解为各个极大团势函数的乘积，再除以归一化常数 Z （**配分函数**）：

$$P(\mathbf{x}) = \frac{1}{Z} \prod_{Q \in \mathcal{C}} \psi_Q(\mathbf{x}_Q)$$

- ψ_Q ：势函数，非负；越大表示这个局部取值「更和谐」。
- Z ：把乘积归一化成合法概率，通常很难算，所以常要近似推断。

三条局部性质：

1. **局部马尔可夫性**：给定邻居，该结点独立于图中其余结点。
2. **成对马尔可夫性**：没有边直接相连的两个结点，给定其他全部结点，二者条件独立。
3. **全局马尔可夫性**：被某个集合隔开的两组变量条件独立。

图像里每个像素一个结点、相邻像素连边：鼓励相邻像素颜色接近，就是 MRF 去噪 / 分割的典型用法。

14.3 条件随机场 CRF

给定观测 $\mathbf{X}$ ，输出标记序列 $\mathbf{Y}$ 构成一张马尔可夫随机场。最常用的是**链式 CRF**，做序列标注（分词、词性、命名实体识别）。

HMM 是**生成式**：建模 $P(\mathbf{X}, \mathbf{Y})$ ，再间接得 $P(\mathbf{Y} | \mathbf{X})$ 。它对观测怎么生成有假设，观测之间的复杂相关不好表达。

CRF 是判别式：直接建模 $P(\mathbf{Y} | \mathbf{X})$ 。给定整句观测，对整条标记序列打分，再 Softmax 成概率。链式 CRF 里输出 $y_1, \dots, y_T$ 构成无向链；势函数可以同时依赖整个观测 $\mathbf{X}$ （前后词都可以看），从而放松 HMM 的独立性约束。

序列标注里，CRF 经常接在神经网络输出后面：网络提供发射分数，CRF 再学标签转移，避免出现非法标签序列。

对照：

- HMM：生成式，建模 $P(\mathbf{X}, \mathbf{Y})$ ，严格马尔可夫假设；
- CRF：判别式，建模 $P(\mathbf{Y} | \mathbf{X})$ ，序列标注通常更优。

14.4 学习与精确推断

- **学习**：估计势函数或条件概率表的参数。
- **推断**：已知部分观测，求未知变量的后验 / 边际。

精确推断两条常用路：

变量消去：按顺序把不需要的变量求和（或积分）掉，得到剩下变量的边缘分布。消元顺序不好时，中间因子会巨大，指数爆炸。

信念传播（消息传递，BP）：结点之间传「我目前对你的看法」。图是树（无环）时精确；有环时变成 loopy BP，只是近似，但实践中常用。

14.5 近似推断

图一大，精确推断不可行。两条大道：

MCMC 采样

构造一条马尔可夫链，转久了以后抽出的样本服从我们想要的分布，再用样本频率近似概率。

- **Metropolis-Hastings**：提议一个新状态，按一定概率接受或拒绝。
- **Gibbs 采样**：每次固定其他变量，按条件分布抽其中一个变量。

缺点：要烧一段时间（burn-in）才算稳态；样本之间相关；难判断收敛。

变分推断

用一个好算的分布 q 去逼近真正的后验 p ，通常最小化 $KL(q||p)$ 。把推断变成优化问题。

深度学习里的 VAE (变分自编码器) 用的 ELBO，就是变分推断：无法直接算后验，就优化一个下界。

14.6 话题模型

LDA (Latent Dirichlet Allocation, 潜在狄利克雷分配) 是三层贝叶斯生成式有向图模型，用来做文本主题挖掘。一篇文档是若干话题的混合；一个话题是词汇表上的一个分布。

例如：某篇新闻 70% 体育 + 30% 财经；「进球」在体育话题里概率高。可做主题发现、文档降维。

注意：这个 LDA 和第 3 章线性判别分析的 LDA 缩写相同、完全不是一回事。

三层随机变量：文档、话题、词。每个文档对应一个话题分布，每个话题对应一个词分布，二者都服从狄利克雷 (Dirichlet) 先验。

生成过程：

- 1. 为文档采样话题分布；
- 2. 文档里每个词：先采样话题，再按该话题采样词语。

学习 / 推断：只观测到词，反推文档-话题、话题-词分布；常用变分推断或 Gibbs 采样。

第 14 章小结

模型	图	生成 / 判别	典型用途
HMM	有向链	生成	序列、语音
MRF	无向	无向联合	图像平滑
CRF	无向 (常线性链)	判别	序列标注
话题 LDA	有向	生成	文档主题

- 1. 概率图模型：结点 = 随机变量，边 = 依赖；有向是贝叶斯网，无向是马尔可夫网 / MRF。
- 2. HMM：三组参数 π, A, B ；三大问题对应前向、Viterbi、EM (前向-后向)；马尔可夫假设。
- 3. MRF：团、极大团、势函数、配分函数 Z ；三条马尔可夫性质；联合概率按极大团分解。
- 4. CRF：判别式，建模 $P(\mathbf{Y} | \mathbf{X})$ ；链式 CRF 做序列标注。
- 5. 推断：精确用变量消去、信念传播 (树上精确)；大规模多用近似 (MCMC / Gibbs、变分)。
- 6. LDA：三层贝叶斯生成模型，狄利克雷先验，挖文本隐主题。

对照：HMM 生成式建模 $P(\mathbf{X}, \mathbf{Y})$ ；CRF 判别式建模 $P(\mathbf{Y} | \mathbf{X})$ 。精确推断适合小图；实际大任务普遍用近似推断。

术语：条件独立、隐状态、转移 / 发射、团 / 极大团、势函数、配分函数、马尔可夫毯、维特比、MCMC、变分推断、话题、狄利克雷。

第 15 章 规则学习

前面多数模型是「一堆权重」，人很难读。规则学习要从数据里学出**人能读懂的逻辑规则**，输出符号化知识，属于符号主义。例如：

若 脐部=凹陷 且 根蒂=蜷缩，则 好瓜。

这和决策树很亲：一棵树可以拆成一组从根到叶的规则。差别是：规则学习往往直接以「一条条规则」为对象，不必先长成一棵树；规则之间可以是列表（按顺序用）、无序集合（投票）等。

两类规则：

- **命题规则**：属性-值逻辑，只能描述简单事实（「色泽=青绿」）。
- **一阶规则**（归纳逻辑程序设计 ILP）：引入谓词和变量，可以描述对象之间的关系，表达能力更强。

15.1 基本概念

规则一般写成 **IF 条件 THEN 结论**。

- **规则体 (body)**：条件部分，由若干文字合取（逻辑与）构成。
- **规则头 (head)**：结论部分，通常表示类别。

例：

IF (根蒂=蜷缩) $\wedge$ (敲声=浊响) THEN 好瓜

一条分类规则也常写成 Horn 子句：

$$\oplus \leftarrow f_1 \wedge f_2 \wedge \cdots \wedge f_L$$

- 左边 $\oplus$ ：结论（正类，或某个类别）。
- 右边：前件，若干**文字**的合取（全部同时成立）。
- **Horn 子句**：后件只有一个正文字，前件是合取。

几个术语：

- **文字 (literal)**：属性=取值，或它的否定。
- **覆盖**：样本满足规则条件，就被这条规则覆盖，规则对它给出预测。希望尽量多覆盖正例、少覆盖负例。
- **纯度**：被覆盖的样本里，有多少真的属于结论那个类。
- **完备**：规则集合覆盖全部样本。
- **一致**：规则集合对所有被覆盖样本都预测正确。

规则学习的目标：生成一组**完备且一致**的规则集合。决策树可以转化成一组规则（根到叶的每条路径一条），但规则学习直接以逻辑规则为对象，不一定是树结构。

两种学习策略：

1. **自底向上 (泛化)**：从具体样例出发，逐步去掉条件、变得更一般。
2. **自顶向下 (特化)**：从很一般的规则开始，逐步增加条件。代表算法：**AQ 算法**。

15.2 序贯覆盖

最常用的命题规则学习框架：**序贯覆盖 (sequential covering)**，一条一条地生成规则。

反复做：

1. 在当前训练集上学**一条**好规则：尽量多覆盖正例、少覆盖反例。
2. 把被它覆盖的样本全部拿走。
3. 在剩下的样本上再学下一条。
4. 直到正例基本都被覆盖。

「学一条规则」本身又常是：从空前件开始贪心加条件（特化），或从太具体的规则开始删条件（泛化）。生成单条规则常用 **FOIL**：用 **FOIL 信息增益** 评估「再加一个文字之后，区分正负样本的能力提升多少」，选增益最大的文字加入规则体。

得到规则集之后，预测可以：

- 按列表从上到下，命中第一条就停（决策表）；
- 所有命中的规则投票；
- 再学一个默认类：谁都没覆盖就猜这个。

序贯覆盖容易**过拟合**，所以需要剪枝。

15.3 剪枝优化

规则一样会过拟合：条件加得太多，只记住几个训练西瓜。评估一条规则常用准确率、覆盖的正例数，或兼顾两者的指标（如 m -估计，给小覆盖量的规则降一点置信）。

可以：

- **预剪枝**：再加一个条件若不能在验证集上变好，就停止。
- **后剪枝**：先学完整规则，再删掉一些文字，看验证性能。

书里常见两种后剪枝：

1. **REP (Reduced-Error Pruning, 减错剪枝)**：数据划成训练集和验证集。先生成完整规则，再在验证集上试着逐个删除规则体里的文字；删了之后验证性能变好，就真删。需要单独划验证集。
2. **PEP (Pessimistic Error Pruning, 悲观剪枝)**：没有独立验证集，直接在训练集上做统计显著性检验，估算剪枝后的泛化表现。

15.4 一阶规则学习

命题规则只能描述单个对象的属性，无法描述**对象之间的关系**。一阶逻辑引入**变量和谓词**，就能写关系。

对照：

- 命题规则：IF (父亲=张三) THEN 儿子=李四
- 一阶规则：IF Father(x , y) THEN Son(y , x)

x, y 是变量，Father($\cdot, \cdot$) 是谓词，表示「 x 是 y 的父亲」。更一般的例子：
祖父(x, z) $\leftarrow$ 父(x, y) $\wedge$ 父(y, z)。

FOIL 也可以用来学一阶规则：仍然用 FOIL 增益，搜索一阶文字。表达能力远强于命题规则，但**假设空间爆炸，计算很贵**。

15.5 归纳逻辑程序设计 ILP

ILP (Inductive Logic Programming) 是一阶规则学习的主流，机器学习与逻辑程序 (Prolog) 的交叉。

输入：背景知识 B 、正例集合、反例集合。

输出：一组 Prolog 逻辑程序（一阶规则集合）。

两大技术路线：

1. **自底向上 / 最小一般泛化 (LGG)**：从具体样例出发，对样例对应的子句做 LGG，得到更通用、但仍覆盖它们的规则。LGG：两条子句最一般的共同泛化。
2. **自顶向下**：从最通用的规则开始，不断特化。
3. **逆归结**：演绎是从一般推特殊；归纳反过来，从特殊例子「倒推」一般规则。

优点：**能用背景知识**，适合关系数据挖掘。

缺点：一阶逻辑假设空间巨大，计算开销极高，只适合小规模、噪声不太大的问题。现实大规模任务很少直接上 ILP。和深度学习结合的方向叫神经符号学习，仍是研究前沿。

第 15 章小结

概念	一句话
Horn 子句	若 A 且 B 且 C，则类别
覆盖 / 完备 / 一致	命中样本；覆盖全体；命中的都判对
序贯覆盖	学一条、拿走样本、再学一条
FOIL 增益	加一个文字后正负分开得更好多少
REP / PEP	减错剪枝（要验证集） / 悲观剪枝（不要）
一阶规则	带变量和谓词，能写关系
ILP	背景知识 + 正反例 → 逻辑程序；LGG / 逆归结

第 16 章 强化学习

前面多数是：给一批固定数据 D ，学一个映射 $f(x) \approx y$ 。

强化学习 (Reinforcement Learning, RL) 换成另一套设定：

智能体在环境里做动作 → 环境给新状态和奖赏 → 再做动作.....目标是让**长期累积奖赏**尽量大。

下棋、玩游戏、机器人走路、广告投放试探，都是这类问题。强化学习研究**智能体 (Agent) 与环境 (Environment) 交互**：智能体执行动作，接收环境反馈的奖赏，学习最优策略，最大化长期累积奖赏。

和监督 / 无监督学习的关键差别：

- **没有给定正确标签**，只有奖赏（可能还稀疏、延迟）；奖赏往往不是执行一步立刻得到，很多时候要等到 episode 结束才知道回报。
- 当前动作会影响以后能看到的数据（探索策略会改变经历）。
- 要在「先试试」和「用已知最好的」之间做权衡。

四大要素：智能体 Agent、环境 Environment、动作 Action、奖赏 Reward。**马尔可夫决策过程 MDP** 是强化学习的理论基础。

16.1 任务与奖赏

标准数学模型：**马尔可夫决策过程 MDP**，记为五元组 $\langle S, A, P, R, \gamma \rangle$ 。

符号	名称	含义
S	状态集	局面、位置、观测
A	动作集	智能体能选的操作
$P(s' s, a)$	转移概率	在 s 做 a 后到 s' 的概率
$R(s, a)$ 或 r	奖赏	这一步的即时反馈
$\gamma \in [0, 1)$	折扣因子	未来的奖赏打几折； γ 接近 1 表示更看长远

马尔可夫性质：下一状态只依赖当前状态和当前动作，不依赖更早历史（历史若重要，应被编码进当前状态）。

一次交互得到一条**轨迹**（回合）： $s_0, a_0, r_1, s_1, a_1, r_2, \dots$ 。

从时刻 t 起的**折扣回报**：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

含义：以后每一拍的奖赏都算上，但越远的奖赏权重越小。 $\gamma = 0$ 就变成只看眼前一步。

两类任务：

- **有限回合 (episodic) 任务**：有终止状态，每个 episode 会结束。回报写到终止时刻 T ：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

- **持续任务 (continuing)**：无终止状态，无限持续，必须用折扣因子避免回报变成无穷大。

策略 π ：在每个状态怎么选动作。可以是确定性的 $a = \pi(s)$ ，也可以是概率 $\pi(a|s)$ 。

两个价值函数：

- **状态价值 $V^\pi(s) = \mathbb{E}[G_t | s_t = s]$** ：从 s 出发按 π 走，长期回报的期望。
- **动作价值 $Q^\pi(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a]$** ：在 s 先做 a ，再按 π 走的长期回报期望。

最优策略 π^* ：使得所有状态下期望累积奖赏最大；对应最优值函数 $V^*(s)$ 、 $Q^*(s, a)$ 。若有 Q^* ，每个状态选 $\arg \max_a Q^*(s, a)$ 就是最优动作。

贝尔曼方程：把值函数分解为**即时奖赏 + 后继状态的期望值**。

16.2 K-摇臂赌博机

简化版 RL：单状态强化学习，只有一个状态， K 个动作（ K 台老虎机），拉哪根臂立刻给奖赏，没有状态转移，不是完整 MDP。用来单独把探索与利用（Exploration-Exploitation）矛盾讲清楚。

- 利用（exploitation）：选当前已知回报最大的动作，最大化当前收益。
- 探索（exploration）：尝试其他动作、收集信息，可能发现更好选择，但会承担短期损失。

只利用会早早锁死在次优臂上；只探索则浪费已经学到的知识。

几种策略

1. ϵ -贪心：以 $1 - \epsilon$ 的概率选当前最优动作（利用），以 ϵ 的概率在所有动作里均匀乱选（探索）。 ϵ 可随时间减小（先多探索，再多利用）。
2. Softmax（玻尔兹曼探索）：按 Q 值做成概率分布再采样， Q 越高被选中概率越大；用温度参数控制探索程度：

$$P(k) = \frac{e^{Q_k/\tau}}{\sum_{i=1}^K e^{Q_i/\tau}}$$

- Q_k ：第 k 臂的当前估计。
- τ ：温度。 τ 大时接近均匀（多探索）， τ 小时接近贪心（多利用）。

这和分类里的 Softmax 是同一个函数，含义换成「选臂的概率」。

16.3 有模型学习

有模型（Model-based）：环境模型已知，也就是状态转移概率 P 和奖赏函数 R 全部已知。不必真去撞环境，可以在脑子里算。代表算法：策略迭代、值迭代。

贝尔曼期望方程（对给定策略 π ）：

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')]$$

读法：当前价值 = 按策略选动作后，「立即奖赏 + 折扣后的下一状态价值」的期望。长期问题被拆成「现在 + 以后」。

动作价值也有对应式子： $Q^\pi(s, a) = \text{立即奖赏} + \text{折扣后下一状态再按 } \pi \text{ 选动作的价值}$ 。

1. 策略迭代

两步循环直到策略收敛：

1. 策略评估： π 固定，求解贝尔曼方程，得到每个状态的 $V^\pi(s)$ 。
2. 策略改进：根据算好的值函数，对每个状态选取能最大化 Q 值的动作 $\arg \max_a Q^\pi(s, a)$ ，得到新策略 π' 。

交替评估-改进，策略单调提升，最终收敛到最优策略。

2. 值迭代

不需要显式维护策略，直接迭代更新最优状态值函数 $V^*(s)$ 。贝尔曼最优方程：

$$V(s) \leftarrow \max_{a \in A} \left\{ R(s, a) + \gamma \sum_{s'} P_{ss'}^a V(s') \right\}$$

迭代收敛后，根据最优 V^* 反推最优策略。

优点：已知环境转移，计算效率高。

缺点：现实绝大多数任务里 P 、 R 未知，无法直接用有模型学习。

16.4 免模型学习

环境的状态转移、奖赏模型未知，智能体只能通过与环境交互得到样本 (s, a, r, s') ，**直接学习策略或者值函数**。两大经典算法：**蒙特卡洛 MC**、**时序差分 TD**。

1. 蒙特卡洛强化学习 MC

基于完整 episode 样本，**必须等到回合结束拿到全部回报，再反过来更新 Q 值**。

采样多条完整轨迹；对每个状态-动作对，计算实际得到的累积回报，用平均回报估计 $Q(s, a)$ 。不需要模型，也不用下一状态的估计。

特点：必须等 episode 结束，属于回合更新；无偏，但方差大；不好直接用于没有终止的持续任务。

2. 时序差分学习 TD（时间差分）

不需要等到回合结束，**每一步就可以更新**，用后继时刻的估计值做引导（**自举 bootstrap**）。

TD 目标： $r + \gamma Q(s', a')$ ，用下一时刻的 Q 估计来当目标。偏差与方差折中。

Sarsa（同策略 on-policy）

同策略：生成样本的策略，和被更新评估的策略是同一个。

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

流程：状态 s 按 ϵ -贪心选 a ；执行动作得到 r, s' ；再从 s' 按同一个策略选出 a' ，然后更新 $Q(s, a)$ 。用的是**自己下一步真会做的 a'** 。Sarsa 更「小心」（会考虑探索时踩到的坑）。

Q-Learning（异策略 off-policy）

异策略：行为策略用来产生样本（ ϵ -贪心做探索），学习优化的却是**最优贪心目标策略**。

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

符号含义：

- α ：学习率，一次补多少误差。
- $r + \gamma \max_{a'} Q(s', a')$ ：TD 目标。
- 括号里：TD 误差，估计偏了多少。

关键点：更新时取 $\max_{a'} Q(s', a')$ ，取后继状态最大 Q ，**不使用实际执行的动作 a'** 。Q-Learning 更「乐观」（按最优后续来更新），直接逼近最优 Q 函数，是最经典的免模型算法。

两者都是表格型方法：每个 (s, a) 存一个数字。

对比：

- **MC**：完整 episode 结束，用真实回报；方差大。
- **TD**：单步更新，用估计值自举；偏差、方差折中。
- **Sarsa (on-policy)**：学的就是产生数据的策略。
- **Q-Learning (off-policy)**：探索采样，学习最优策略。

16.5 值函数近似

前面 Sarsa / Q-Learning 都是**表格型强化学习**：每个 (s, a) 维护独立 Q 值表。

当状态空间巨大、连续（如机器人传感器、图像），表格存不下，需要**函数近似**：不再把 $Q(s, a)$ 存成表，而是用带参数的模型近似，例如线性模型、神经网络：

$$Q(s, a; \boldsymbol{w}) \approx Q^*(s, a)$$

把强化学习变成类似监督学习：把 TD 目标当作标签，最小化损失，对参数 $\boldsymbol{w}$ 做梯度下降。

注意：函数近似会出现训练不稳定、发散；目标值本身是网络自己预测的，目标不断变化，存在**自举偏差**。

DQN (Deep Q-Network)：Q-Learning + 深度神经网络 + 经验回放 + 目标网络，用来缓解不稳定。这是深度 RL 的入门接口：西瓜书的 Q-learning + 第 5 章的神经网络。

16.6 模仿学习

不一定自己从零试错（试错在现实里可能摔坏机器人）。可以模仿专家。

- **行为克隆**：把专家的 (s, a) 当监督学习数据，直接学 $\pi(s) \approx a_{\text{专家}}$ 。简单，但专家没去过的状态会崩（**分布偏移 / 分布漂移**）。
- **逆强化学习 (IRL)**：假设专家在优化某个未知奖赏，先从专家轨迹反推奖赏函数，再对这个奖赏做普通 RL。适合「奖赏很难手写、但专家示范好拿」的任务。

第 16 章小结

概念	记住
MDP	五元组 $\langle S, A, P, R, \gamma \rangle$ ；马尔可夫性质
V / Q	状态好不好 / 在某状态做某动作好不好；贝尔曼方程
探索-利用	ϵ -贪心、Softmax；摇臂赌博机不是完整 MDP
有模型	知道 P, R ；策略迭代 / 值迭代
免模型	MC 等回合结束；TD 用自举估计
Sarsa / Q-learning	同策略用真实 a' / 异策略用 $\max_{a'} Q(s', a')$
函数近似 / DQN	用网络代替 Q 表；注意自举偏差

概念	记住
模仿学习	行为克隆（分布漂移）；IRL 反推奖赏

1. 强化学习：Agent 与环境交互，最大化**长期累积折扣奖赏**；MDP 五元组 $\langle S, A, P, R, \gamma \rangle$ ；马尔可夫性质；值函数 V 、 Q ，贝尔曼方程；最优策略。
2. K 摇臂赌博机：**探索-利用的矛盾**； ϵ -贪心、Softmax。
3. 有模型学习（ P 已知）：策略迭代（策略评估 + 策略改进）；值迭代。
4. 免模型学习（环境未知）
 - MC 蒙特卡洛：完整 episode 结束，用真实回报更新；
 - TD 时序差分：单步自举更新；
 - Sarsa（同策略 on-policy）；Q-Learning（异策略 off-policy）。
5. 值函数近似：状态空间太大时，用线性 / 神经网络代替 Q 表格；带来训练不稳定。
6. 模仿学习：行为克隆（监督模仿，分布漂移）；逆强化学习 IRL 反推奖赏函数。

关键区分：

- **有模型**：知道环境转移 P ；**免模型**：只靠交互采样。
- **On-policy**：产生数据和学习是同一个策略；**Off-policy**：采样策略和学习策略不同。

术语：智能体、环境、奖赏、策略、折扣回报、on-policy / off-policy、TD 误差、自举、模仿学习。
